



bok25

基礎

后綴关注公众号[研大]
永久微信1951657888

課、操作系統





同步与互斥

不可再现性：进程在并发执行时，由于失去了封闭性，也将导致其再失去可再现性。

```
x=100  
x++  
printf("%d",x)
```

程序A

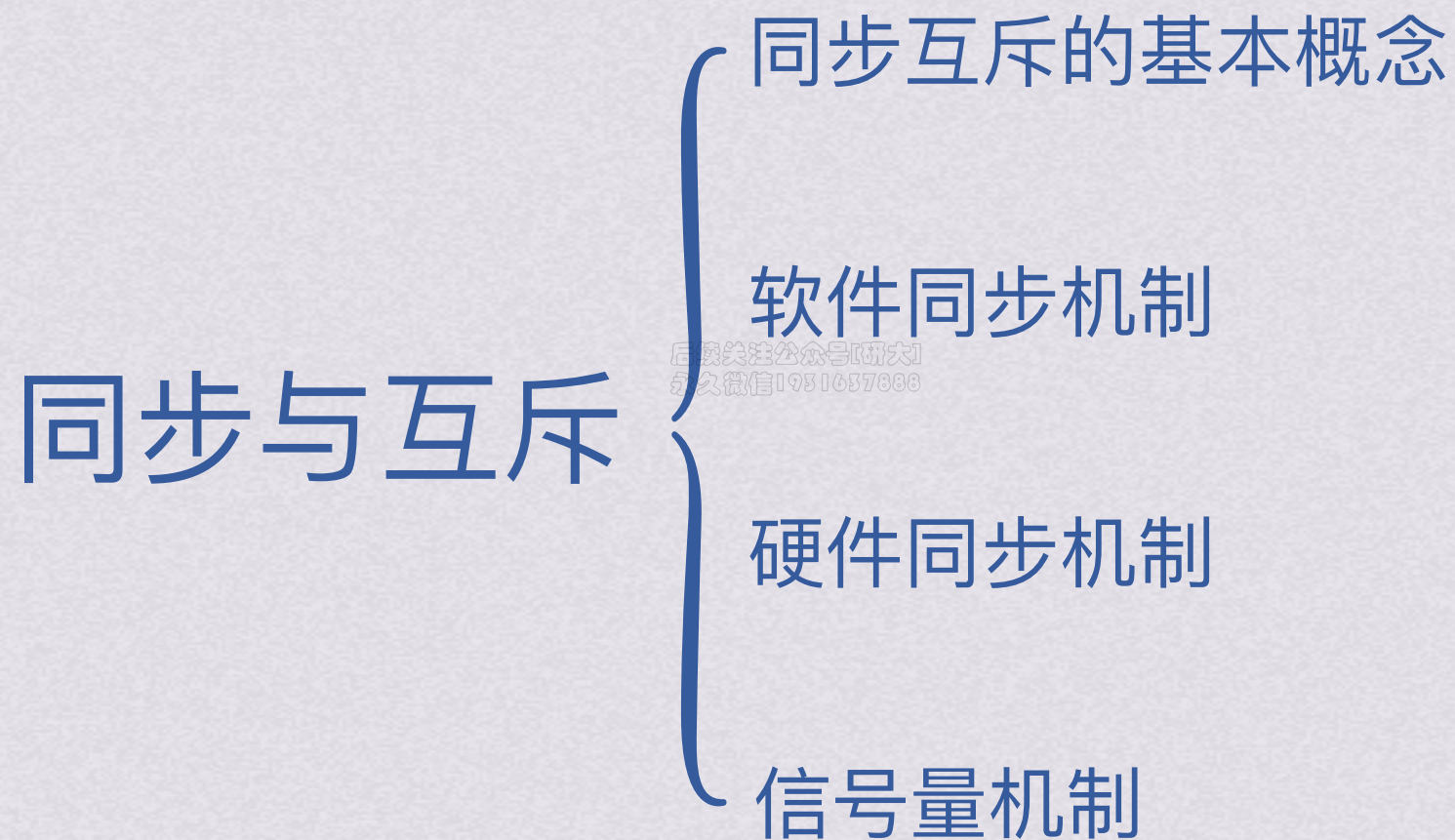
X=5

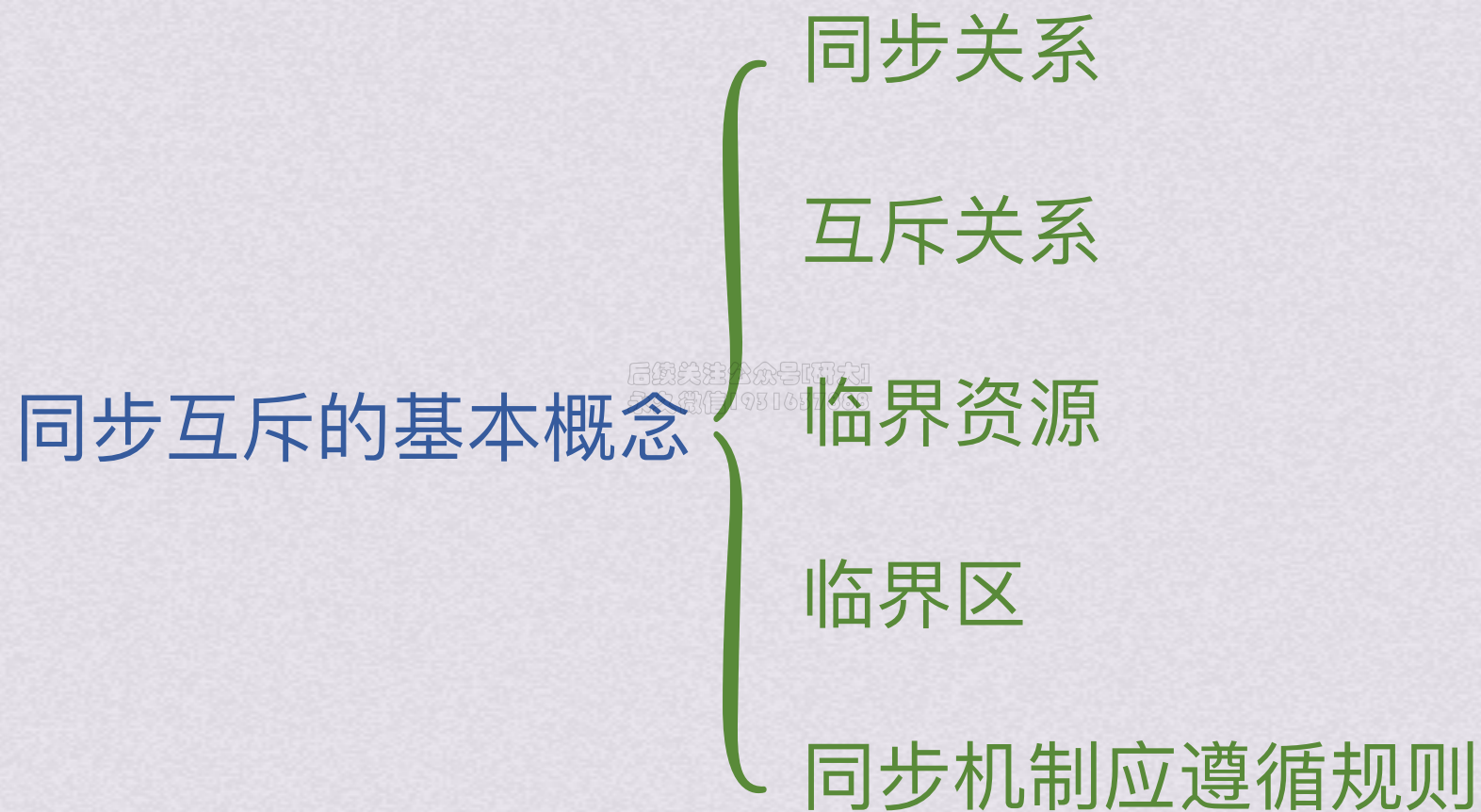
内存

```
x=5  
x--  
printf("%d",x)
```

程序B







后续关注公众号[研木]
微信:1951651183





同步互斥的基本概念

同步关系

假设有输入进程A与计算进程B，进程A需要给进程B通过共享的缓冲区提供数据。这就有了缓冲区满时A不能投放数据而阻塞；缓冲区空时B因不能取出数据而阻塞。这样直接相互制约的关系称为同步关系





同步互斥的基本概念

同步关系

假设有输入进程A与计算进程B，进程A需要给进程B通过共享的缓冲区提供数据。这就有了缓冲区满时A不能投放数据而阻塞；缓冲区空时B因不能取出数据而阻塞。这样直接相互制约的关系称为同步关系





同步互斥的基本概念

同步关系

假设有输入进程A与计算进程B，进程A需要给进程B通过共享的缓冲区提供数据。这就有了缓冲区满时A不能投放数据而阻塞；缓冲区空时B因不能取出数据而阻塞。这样直接相互制约的关系称为同步关系



- 互斥关系
- 临界资源
- 临界区
- 同步机制应遵循规则
- 软件同步机制
- 硬件同步机制
- 信号量机制

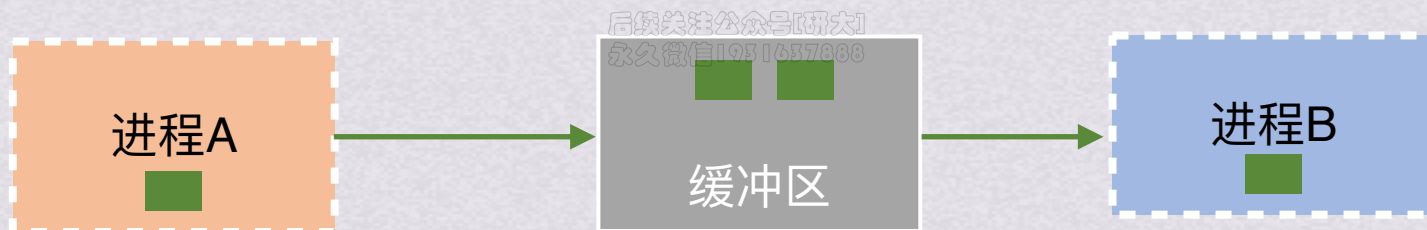




同步互斥的基本概念

同步关系

假设有输入进程A与计算进程B，进程A需要给进程B通过共享的缓冲区提供数据。这就有了缓冲区满时A不能投放数据而阻塞；缓冲区空时B因不能取出数据而阻塞。这样直接相互制约的关系称为同步关系



- 互斥关系
- 临界资源
- 临界区
- 同步机制应遵循规则
- 软件同步机制
- 硬件同步机制
- 信号量机制

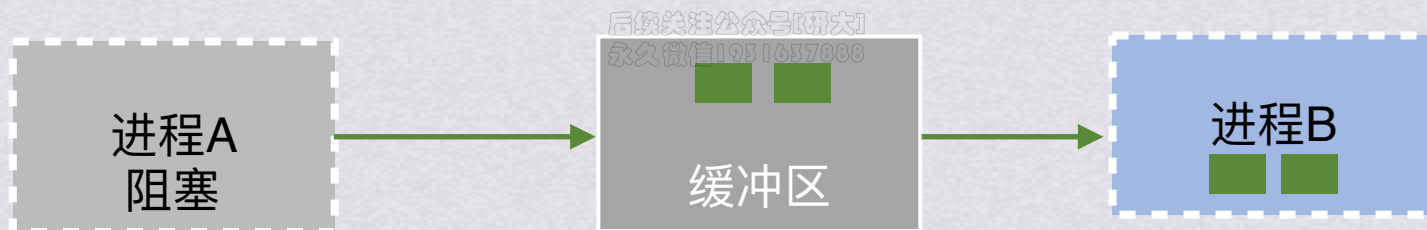




同步互斥的基本概念

同步关系

假设有输入进程A与计算进程B，进程A需要给进程B通过共享的缓冲区提供数据。这就有了缓冲区满时A不能投放数据而阻塞；缓冲区空时B因不能取出数据而阻塞。这样直接相互制约的关系称为同步关系



- 互斥关系
- 临界资源
- 临界区
- 同步机制应遵循规则
- 软件同步机制
- 硬件同步机制
- 信号量机制





同步互斥的基本概念

同步关系

假设有输入进程A与计算进程B，进程A需要给进程B通过共享的缓冲区提供数据。这就有了缓冲区满时A不能投放数据而阻塞；缓冲区空时B因不能取出数据而阻塞。这样直接相互制约的关系称为同步关系



- 互斥关系
- 临界资源
- 临界区
- 同步机制应遵循规则
- 软件同步机制
- 硬件同步机制
- 信号量机制





同步互斥的基本概念

同步关系

- 互斥关系
- 临界资源
- 临界区
- 同步机制应遵循规则
- 软件同步机制
- 硬件同步机制
- 信号量机制





同步互斥的基本概念

互斥关系

- 同步关系
- 临界资源
- 临界区
- 同步机制应遵循规则
- 软件同步机制
- 硬件同步机制
- 信号量机制

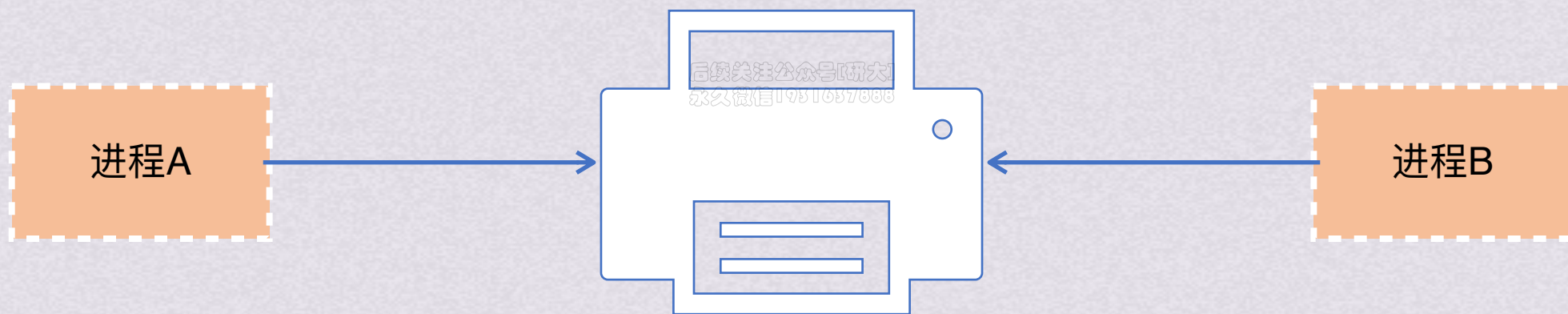




同步互斥的基本概念

互斥关系

多个程序并发执行时，对于像打印机这样的临界资源，必须保证多个进程对它只能互斥的访问。所以这类资源必须用户使用前先提出申请，再由系统实施统一分配。这样间接相互制约的关系称为互斥关系



- 同步关系
- 临界资源
- 临界区
- 同步机制应遵循规则
- 软件同步机制
- 硬件同步机制
- 信号量机制

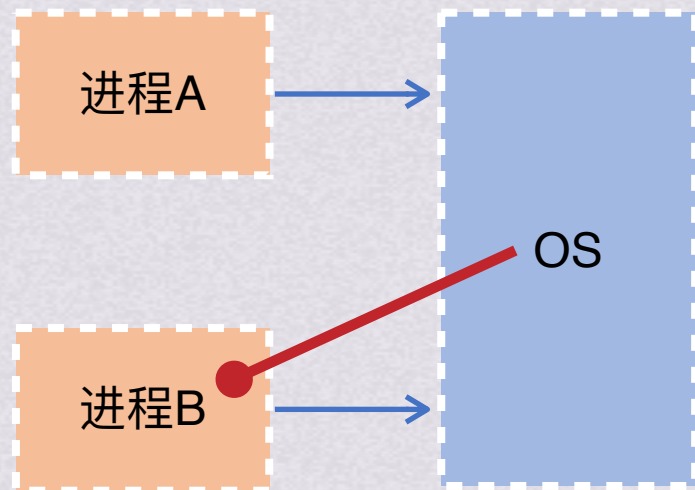




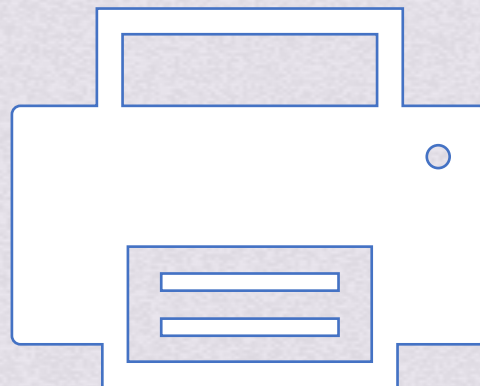
同步互斥的基本概念

互斥关系

多个程序并发执行时，对于像打印机这样的临界资源，必须保证多个进程对它只能互斥的访问。所以这类资源必须用户使用前先提出申请，再由系统实施统一分配。这样间接相互制约的关系称为互斥关系



后续关注公众号[研大]
永久微信1951657888



- 同步关系
- 临界资源
- 临界区
- 同步机制应遵循规则
- 软件同步机制
- 硬件同步机制
- 信号量机制

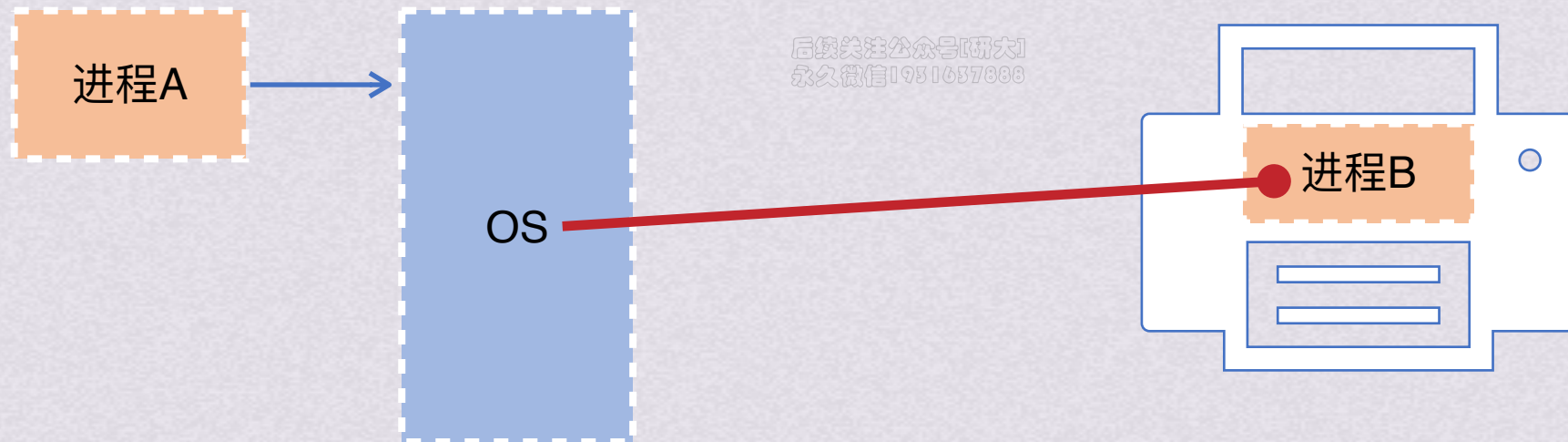




同步互斥的基本概念

互斥关系

多个程序并发执行时，对于像打印机这样的临界资源，必须保证多个进程对它只能互斥的访问。所以这类资源必须用户使用前先提出申请，再由系统实施统一分配。这样间接相互制约的关系称为互斥关系



- 同步关系
- 临界资源
- 临界区
- 同步机制应遵循规则
- 软件同步机制
- 硬件同步机制
- 信号量机制





同步互斥的基本概念

互斥关系

- 同步关系
- 临界资源
- 临界区
- 同步机制应遵循规则
- 软件同步机制
- 硬件同步机制
- 信号量机制





同步互斥的基本概念

临界资源

- 同步关系
- 互斥关系
- 临界区
- 同步机制应遵循规则
- 软件同步机制
- 硬件同步机制
- 信号量机制





同步互斥的基本概念

临界资源

在一段时间内只允许一个进程访问的资源，称为临界资源（或独占资源）
对于临界资源的共享，各进程之间应该采用互斥方式

```
void producer(){  
    while(1){  
        produce;  
        while(counter == n);  
        buffer[in] = nextp;  
        in =(in+1) % n;  
        counter++;  
    }  
};
```

进程A

后续关注公众号[研大]
永久微信1951657888

缓冲区

进程B

```
void consumer(){  
    while(1){  
        while(counter == 0);  
        nextc = buffer[out];  
        out =(out+1) % n;  
        counter--;  
    }  
};
```

看起来没有问题的代码，执行中为什么会出错？





同步互斥的基本概念

临界资源

在一段时间内只允许一个进程访问的资源，称为临界资源（或独占资源）
对于临界资源的共享，各进程之间应该采用互斥方式

看起来没有问题的代码，执行中为什么会出错？

counter++

```
register1 = counter;  
register1 = register1 + 1;  
counter = register1;
```

```
void producer(){  
    while(1){  
        produce;  
        while(counter == n);  
        buffer[in] = nextp;  
        in =(in+1) % n;  
        counter++;  
    }  
};
```

```
void consumer(){  
    while(1){  
        while(counter == 0);  
        nextc = buffer[out];  
        out =(out+1) % n;  
        counter--;  
    }  
};
```

counter--

```
register2 = counter;  
register2 = register2 - 1;  
counter = register2;
```





同步互斥的基本概念

临界资源

在一段时间内只允许一个进程访问的资源，称为临界资源（或独占资源）
对于临界资源的共享，各进程之间应该采用互斥方式

看起来没有问题的代码，执行中为什么会出错？

counter++

```
register1 = counter;  
register1 = register1 + 1;  
counter = register1;
```

counter--

```
register2 = counter;  
register2 = register2 - 1;  
counter = register2;
```

counter = 2,并发执行counter++,counter--

```
register1=counter (register1=2)  
register1=register1 + 1 (register1=3)
```

永久微信1951657888

```
register2 = counter (register2 = 2)  
register2 = register 2 - 1 (register 2 = 1)
```

```
counter = register1 (counter = 3)
```

```
counter = register2 (counter = 1)
```

counter=2，执行了counter++和counter--后，变成了counter=1

解决方法：把counter当做临界资源处理，令进程互斥地访问变量counter





同步互斥的基本概念

临界资源

- 同步关系
- 互斥关系
- 临界区
- 同步机制应遵循规则
- 软件同步机制
- 硬件同步机制
- 信号量机制





同步互斥的基本概念

临界区

- 同步关系
- 互斥关系
- 临界资源
- 同步机制应遵循规则
- 软件同步机制
- 硬件同步机制
- 信号量机制





同步互斥的基本概念

临界区

不管硬件临界资源还是软件的临界资源，多个进程必须互斥的访问。

每个进程进入临界区之前需要先检查临界资源是否正在被访问，这段代码称为进入区

临界区后面也要加上一段退出区代码，用于将临界区正被访问的标志恢复

```
while(TRUE){  
    进入区  
    临界区  
    退出区  
    剩余区  
};
```

每个进程中访问临界资源的那段代码称为临界区

进程中除了这三部分，其他代码称为剩余区





同步互斥的基本概念

临界区

- 同步关系
- 互斥关系
- 临界资源
- 同步机制应遵循规则
- 软件同步机制
- 硬件同步机制
- 信号量机制





同步互斥的基本概念

同步机制应遵循规则

- 同步关系
- 互斥关系
- 临界资源
- 临界区
- 软件同步机制
- 硬件同步机制
- 信号量机制





同步互斥的基本概念

同步机制应遵循规则

- 1.空闲让进:** 无进程处于临界区时, 表明临界资源空闲, 应允许一个请求进入临界区的进程立即进入自己的临界区, 有效利用资源
- 2.忙则等待:** 已有进程进入临界区时, 表明临界资源正在被访问, 因而其他试图进入临界区的进程必须等待, 以保证对临界资源的互斥访问
- 3.有限等待:** 对要求访问临界资源的进程, 应保证在有限时间内能进入自己的临界区, 避免陷入“死等”状态
- 4.让权等待:** 进程不能进入自己的临界区时, 应立即释放处理机, 以免进程陷入“忙等”状态

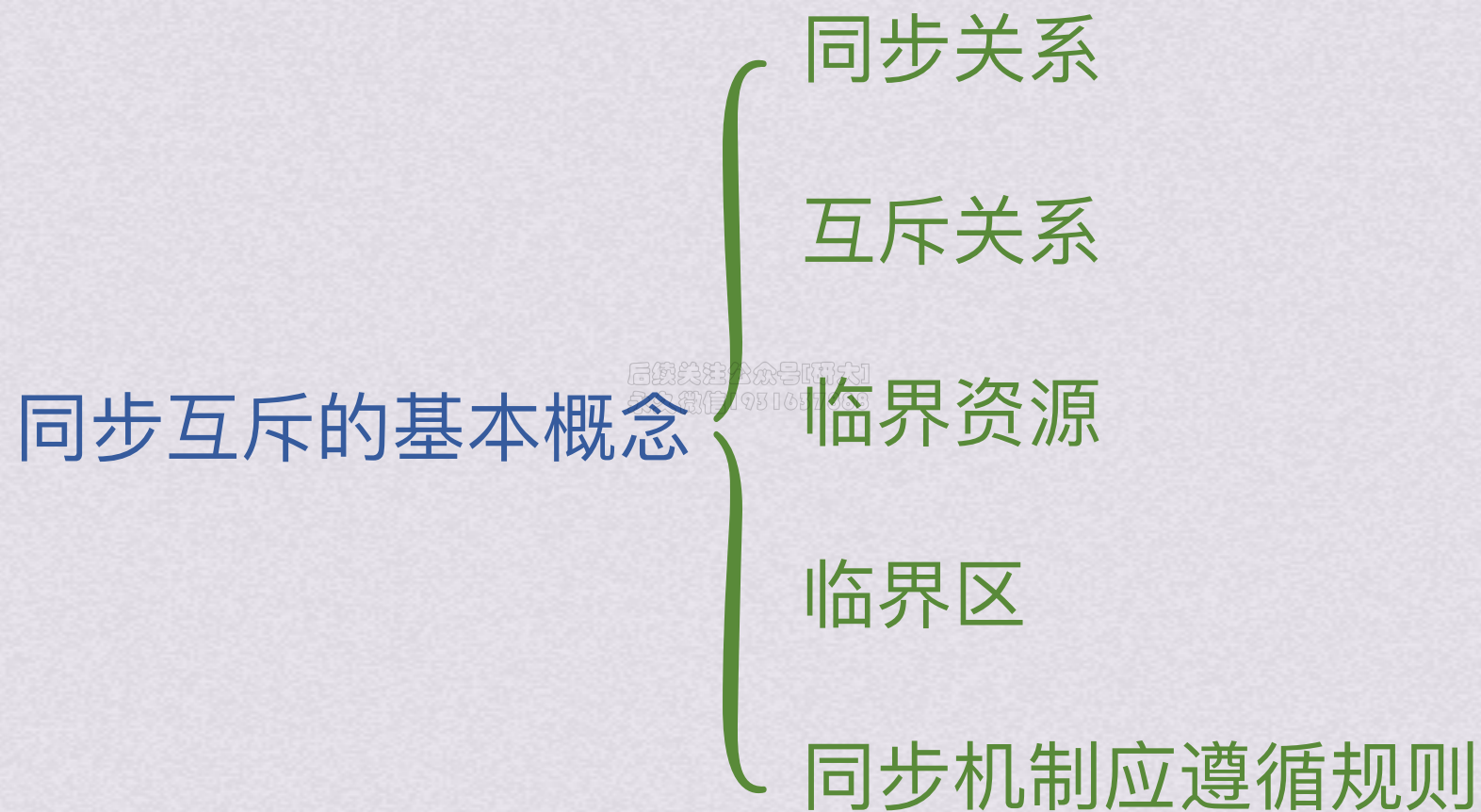
后续关注公众号[研大]
永久微信1951657888

朋友: 你到哪了?
我: 在等车 IO
朋友: 那你快点!
我:



快速等待





后续关注公众号[研木]
微信号: 1951651183





软件同步机制





软件同步机制

单标志法

双标志先检查法

双标志后检查法

peterson算法

扫码关注公众号「研大」
永久微信 1951637888





软件同步机制

单标志法

设置一公用整型变量来标明是否有进程在临界区中，如果已有进程在临界区，则在进入区通过循环检查进行等待，进程离开临界区后则在退出区修改标志

单标志法设置一个公用整型变量turn，指示允许进入临界区的进程编号，turn=0时允许进程P0进入临界区；turn=1时运行进程P1进入临界区，退出临界区时将临界区使用权赋予另一进程（Pi退出临界区时，令turn=j）

```
//进程P0
while(turn!=0);    //进入区
critical section; //临界区
turn = 1;          //退出区
remainder section; //剩余区
```

后续关注公众号[研大]
永久微信1951657888

```
//进程P1
while(turn!=1);    //进入区
critical section; //临界区
turn = 0;          //退出区
remainder section; //剩余区
```

双标志先检查法
双标志后检查法
peterson算法

同步互斥的基本概念

硬件同步机制

信号量机制





软件同步机制

```
//进程P0
while(turn!=0); //进入区
critical section; //临界区
turn = 1; //退出区
remainder section; //剩余区

//进程P1
while(turn!=1); //进入区
critical section; //临界区
turn = 0; //退出区
remainder section; //剩余区
```

单标志法

单标志法设置一个公用整型变量turn，指示允许进入临界区的进程编号，turn=0时允许进程P0进入临界区；turn=1时运行进程P1进入临界区，退出临界区时将临界区使用权赋予另一进程（Pi退出临界区时，令turn=j）

这是1车道



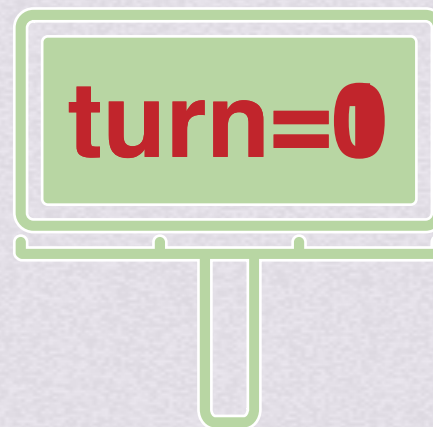
临界区

车一旦撞线就更改turn的数值

进入区



这是0车道



turn的数字为可以通行的车道

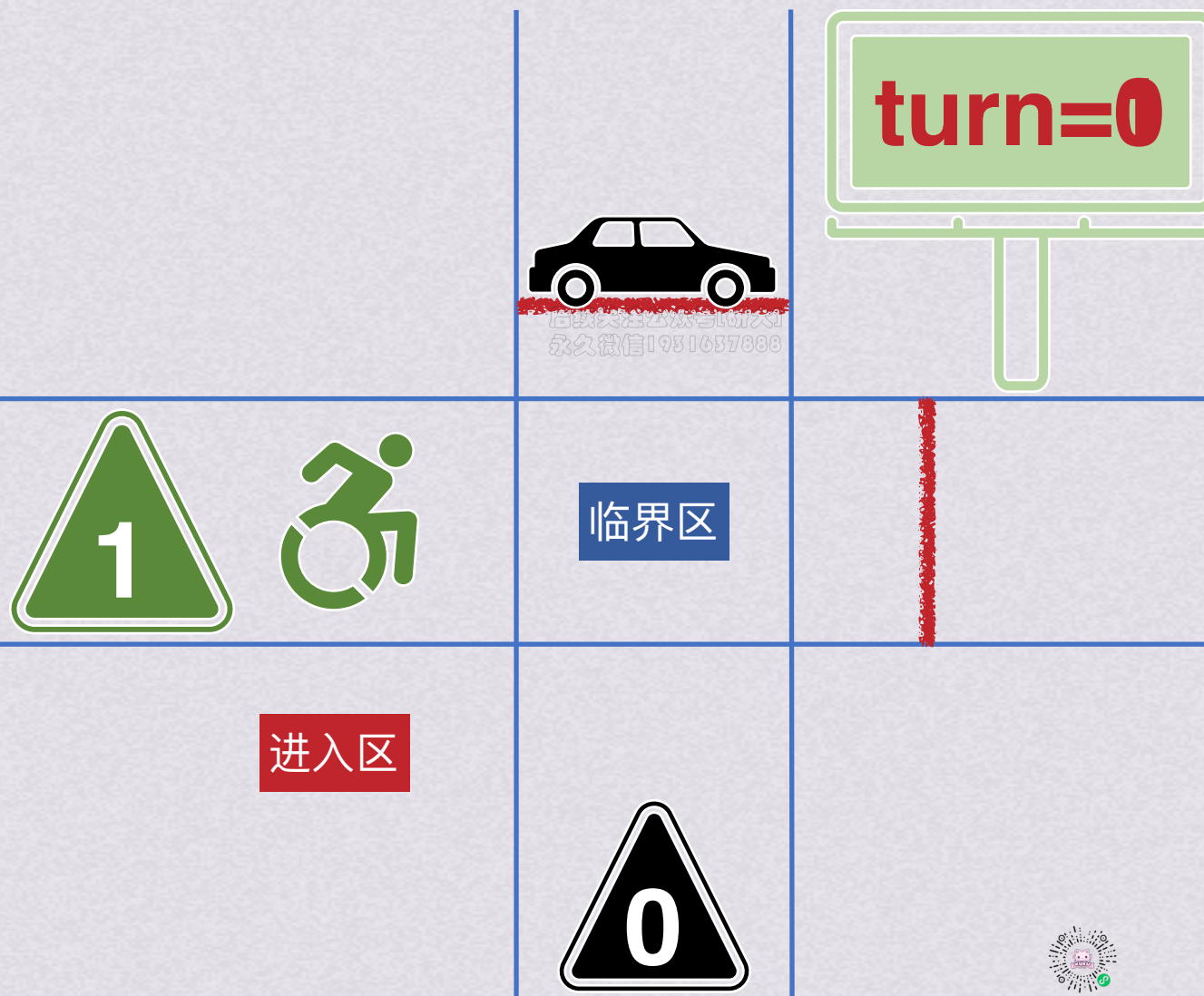
双标志先检查法
双标志后检查法
peterson算法
同步互斥的基本概念
硬件同步机制
信号量机制



单标志法

```
//进程P1
while(turn!=1);    //进入区
critical section;  //临界区
turn = 0;          //退出区
remainder section; //剩余区
```

单标志法设置一个公用整型变量turn，指示允许进入临界区的进程编号，turn=0时允许进程P0进入临界区；turn=1时运行进程P1进入临界区，退出临界区时将临界区使用权赋予另一进程（Pi退出临界区时，令turn=j）



- 双标志先检查法
- 双标志后检查法
- peterson算法
- 同步互斥的基本概念
 - 硬件同步机制
 - 信号量机制



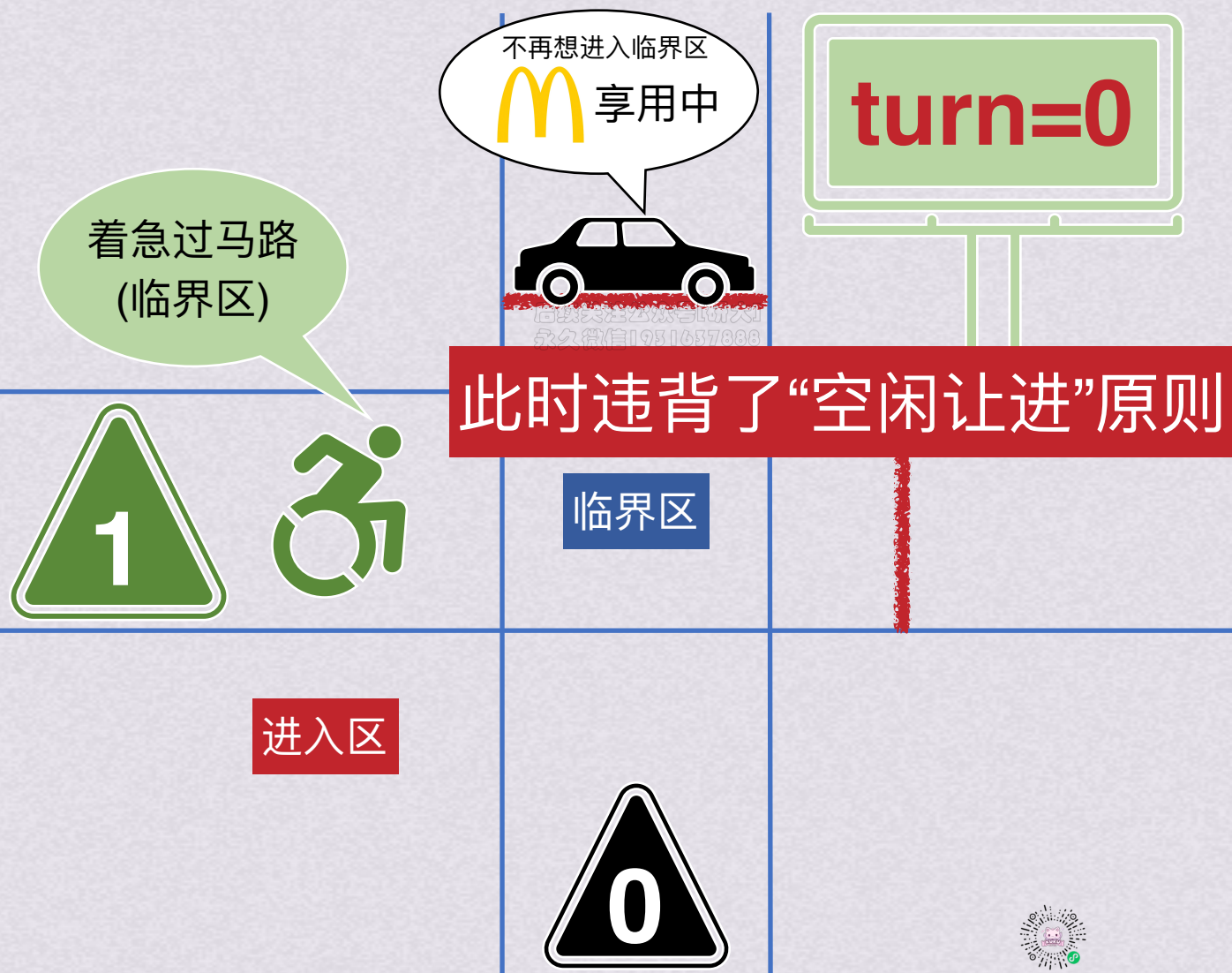
软件同步机制

```
//进程P0  
while(turn!=0); //进入区  
critical section; //临界区  
turn = 1; //退出区  
remainder section; //剩余区
```

```
//进程P1  
while(turn!=1); //进入区  
critical section; //临界区  
turn = 0; //退出区  
remainder section; //剩余区
```

单标志法

单标志法设置一个公用整型变量turn，指示允许进入临界区的进程编号，turn=0时允许进程P0进入临界区；turn=1时运行进程P1进入临界区，退出临界区时将临界区使用权赋予另一进程（Pi退出临界区时，令turn=j）



- 双标志先检查法
- 双标志后检查法
- peterson算法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制





软件同步机制

单标志法

单标志法设置一个公用整型变量turn，指示允许进入临界区的进程编号，turn=0时允许进程P0进入临界区；turn=1时运行进程P1进入临界区，退出临界区时将临界区使用权赋予另一进程（Pi退出临界区时，令turn=j）

```
//进程P0
while(turn!=0);    //进入区
critical section;  //临界区
turn = 1;          //退出区
remainder section; //剩余区
```

```
//进程P1
while(turn!=1);    //进入区
critical section;  //临界区
turn = 0;          //退出区
remainder section; //剩余区
```

两个进程必须交替进入临界区，若一个进程不再进入临界区，则另一个进程也无法进入临界区

违背“空闲让进”准则

双标志先检查法
双标志后检查法
peterson算法

同步互斥的基本概念

硬件同步机制

信号量机制





单标志法

双标志先检查法
双标志后检查法
peterson算法

同步互斥的基本概念
硬件同步机制
信号量机制





软件同步机制

双标志先检查法

- 单标志法
- 双标志后检查法
- peterson算法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制





软件同步机制

双标志先检查法

双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false

```
//进程P0
while(flag[1]);    //进入区
flag[0]=true;     //进入区
critical section; //临界区
flag[0]=false;    //退出区
remainder section; //剩余区
```

```
//进程P1
while(flag[0]);    //进入区
flag[1]=true;     //进入区
critical section; //临界区
flag[1]=false;    //退出区
remainder section; //剩余区
```





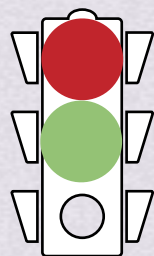
软件同步机制

双标志先检查法

```
//进程P0
while(flag[1]); //进入区
flag[0]=true; //进入区
critical section; //临界区
flag[0]=false; //退出区
remainder section; //剩余区
```

```
//进程P1
while(flag[0]); //进入区
flag[1]=true; //进入区
critical section; //临界区
flag[1]=false; //退出区
remainder section; //剩余区
```

双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false

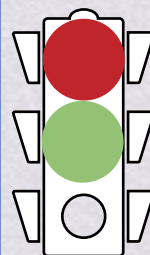


后浪关注公众号「研大」
永久微信1951657888

临界区

压绿线就把别人的红绿灯改成绿色
对应flag[i]=false

压红线就把别人的红绿灯改成红色
对应flag[i]=true



单标志法
双标志后检查法
peterson算法
同步互斥的基本概念
硬件同步机制
信号量机制





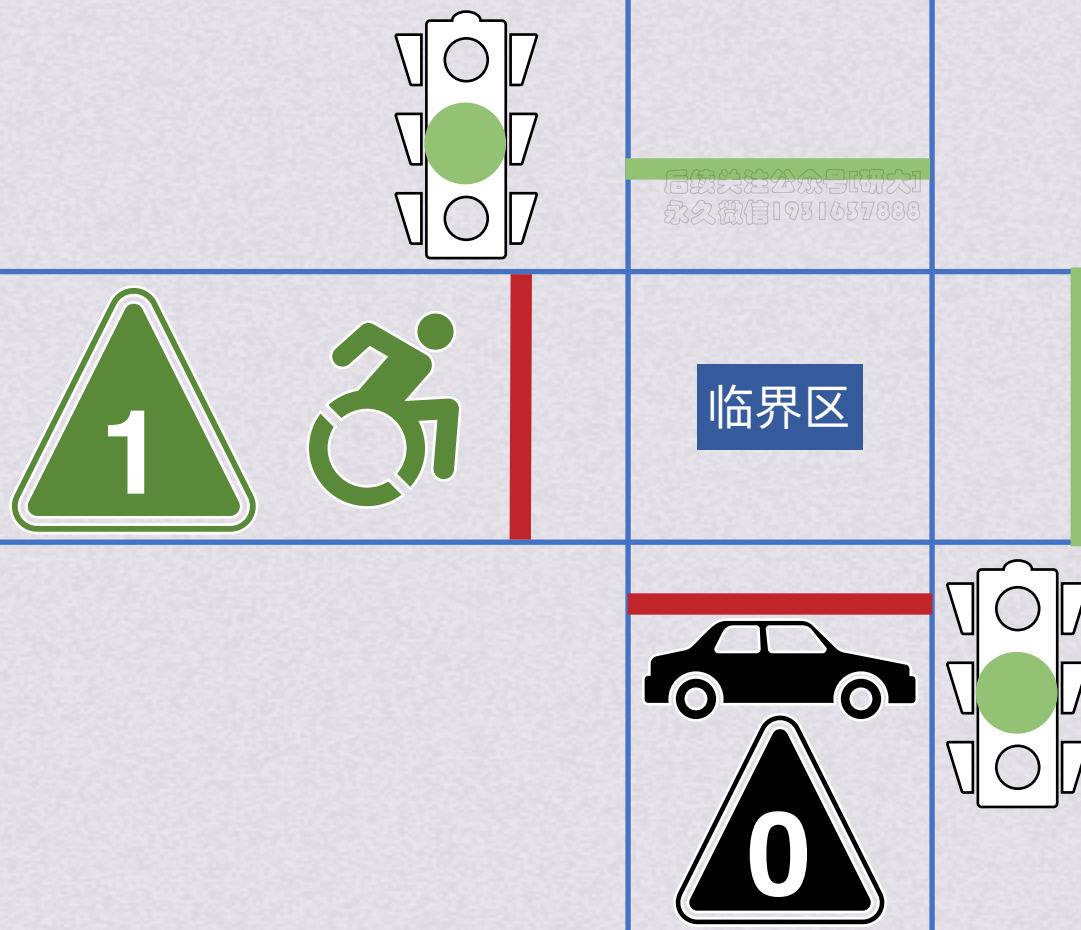
软件同步机制

双标志先检查法

```
//进程P0  
while(flag[1]); //进入区  
flag[0]=true; //进入区  
critical section; //临界区  
flag[0]=false; //退出区  
remainder section; //剩余区
```

```
//进程P1  
while(flag[0]); //进入区  
flag[1]=true; //进入区  
critical section; //临界区  
flag[1]=false; //退出区  
remainder section; //剩余区
```

双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false



- 单标志法
- 双标志后检查法
- peterson算法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制





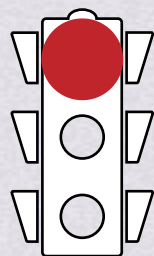
软件同步机制

双标志先检查法

```
//进程P0  
while(flag[1]); //进入区  
flag[0]=true; //进入区  
critical section; //临界区  
flag[0]=false; //退出区  
remainder section; //剩余区
```

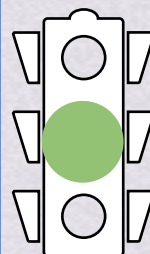
```
//进程P1  
while(flag[0]); //进入区  
flag[1]=true; //进入区  
critical section; //临界区  
flag[1]=false; //退出区  
remainder section; //剩余区
```

双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false



后浪关注公众号「研大」
永久微信1951657888

临界区



- 单标志法
- 双标志后检查法
- peterson算法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制





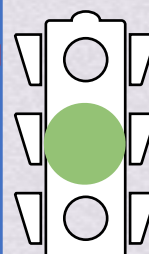
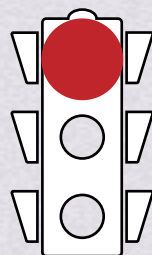
软件同步机制

双标志先检查法

```
//进程P0  
while(flag[1]); //进入区  
flag[0]=true; //进入区  
critical section; //临界区  
flag[0]=false; //退出区  
remainder section; //剩余区
```

```
//进程P1  
while(flag[0]); //进入区  
flag[1]=true; //进入区  
critical section; //临界区  
flag[1]=false; //退出区  
remainder section; //剩余区
```

双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false



后浪关注公众号「研大」
永久微信1951657888

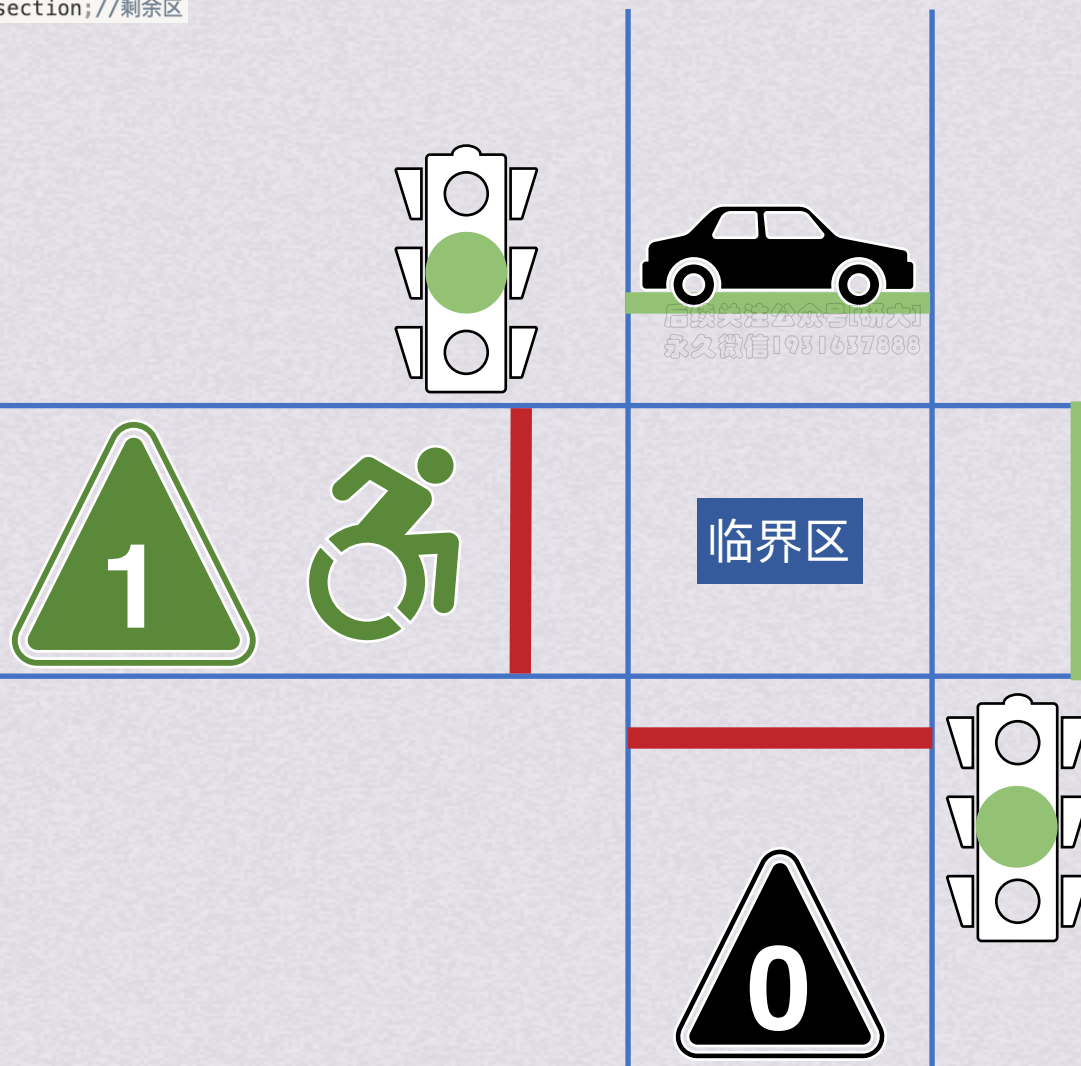
- 单标志法
- 双标志后检查法
- peterson算法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



双标志先检查法

```
//进程P1
while(flag[0]);    //进入区
flag[1]=true;      //进入区
critical section;  //临界区
flag[1]=false;     //退出区
remainder section; //剩余区
```

双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false



- 单标志法
- 双标志后检查法
- peterson算法
- 同步互斥的基本概念
 - 硬件同步机制
 - 信号量机制





软件同步机制

双标志先检查法

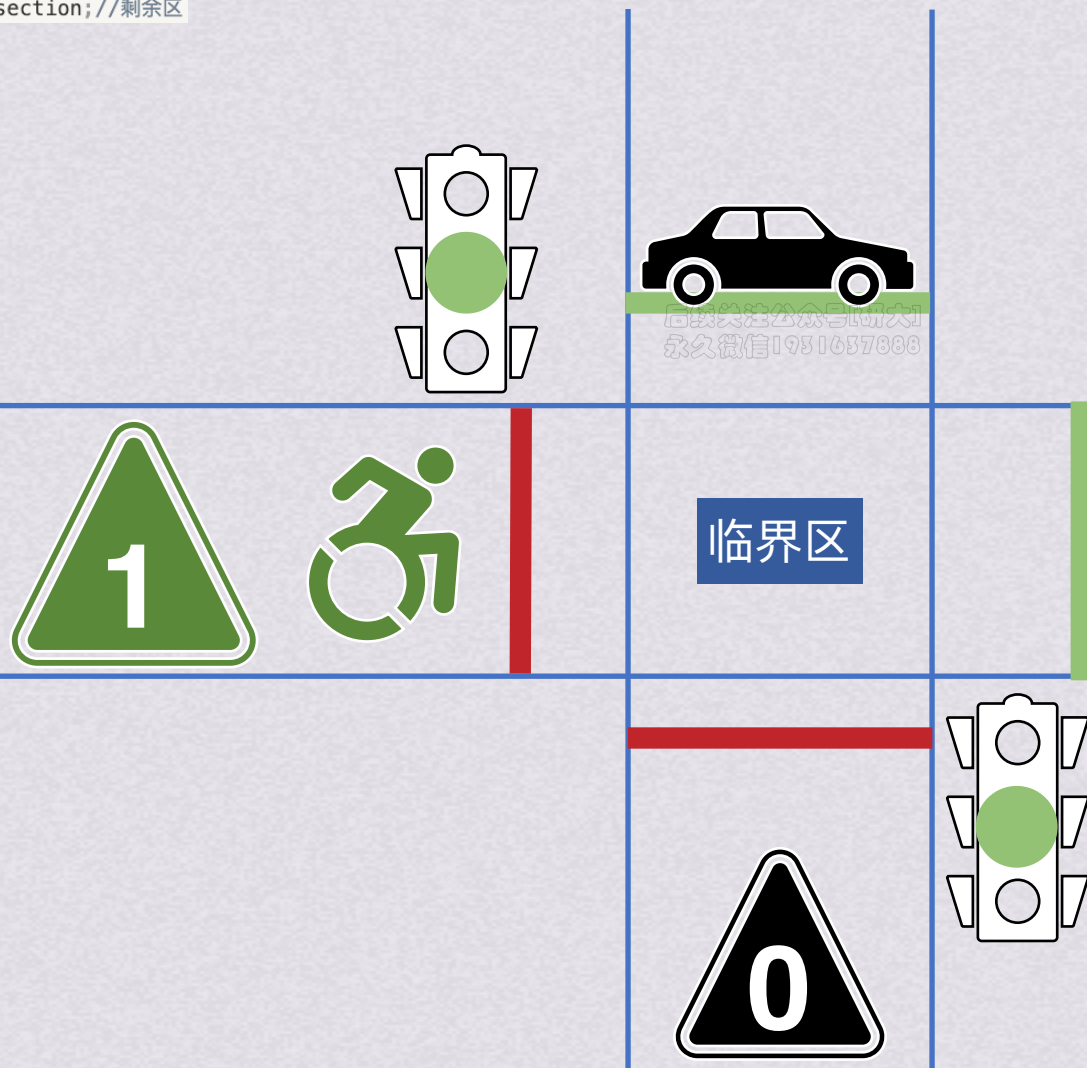
```
//进程P0  
while(flag[1]); //进入区  
flag[0]=true; //进入区  
critical section; //临界区  
flag[0]=false; //退出区  
remainder section; //剩余区
```

```
//进程P1  
while(flag[0]); //进入区  
flag[1]=true; //进入区  
critical section; //临界区  
flag[1]=false; //退出区  
remainder section; //剩余区
```

双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。

Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；

否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false



- 单标志法
- 双标志后检查法
- peterson算法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制





软件同步机制

双标志先检查法

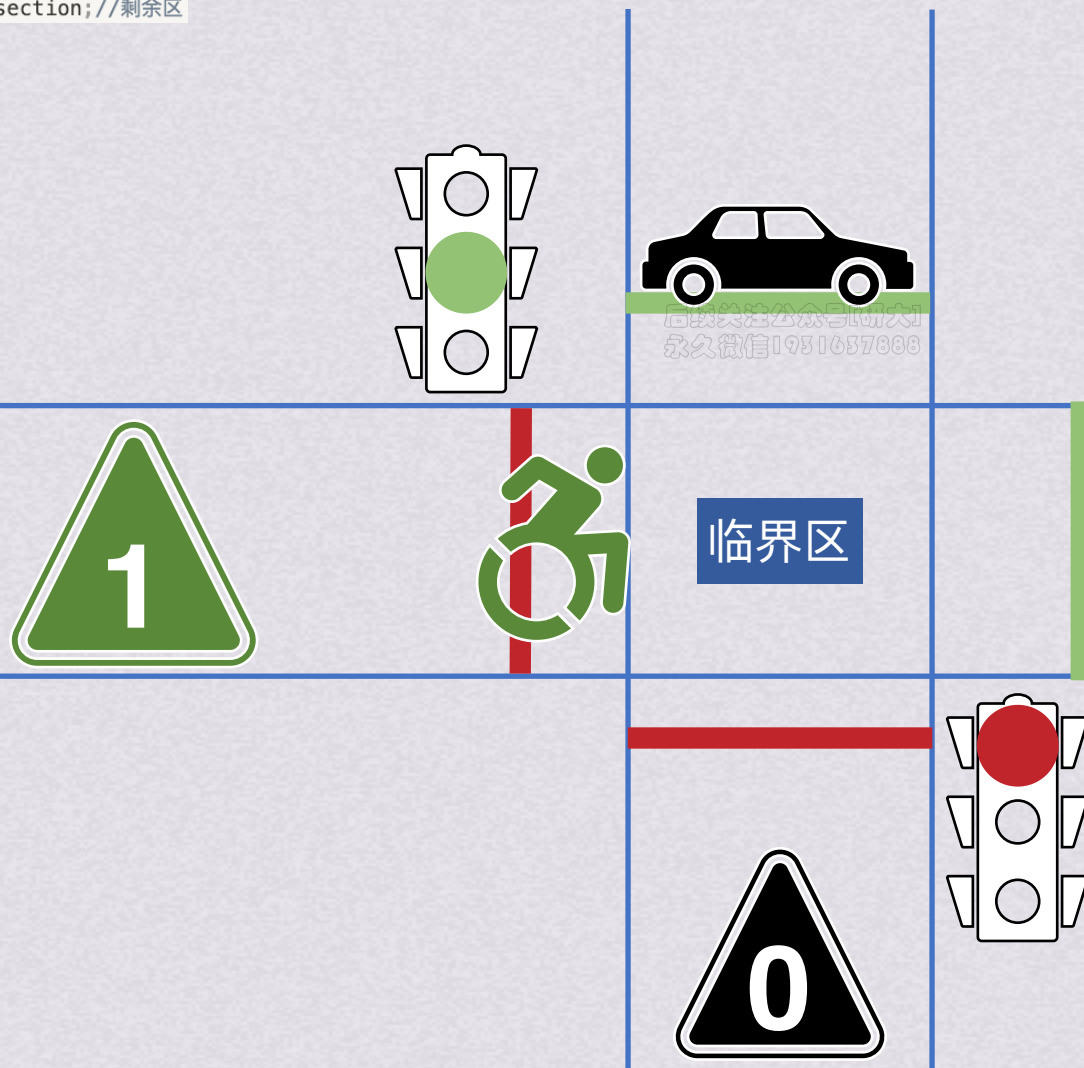
```
//进程P0  
while(flag[1]); //进入区  
flag[0]=true; //进入区  
critical section; //临界区  
flag[0]=false; //退出区  
remainder section; //剩余区
```

```
//进程P1  
while(flag[0]); //进入区  
flag[1]=true; //进入区  
critical section; //临界区  
flag[1]=false; //退出区  
remainder section; //剩余区
```

双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。

Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；

否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false



- 单标志法
- 双标志后检查法
- peterson算法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制





软件同步机制

双标志先检查法

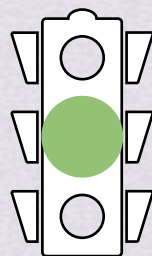
```
//进程P0  
while(flag[1]); //进入区  
flag[0]=true; //进入区  
critical section; //临界区  
flag[0]=false; //退出区  
remainder section; //剩余区
```

```
//进程P1  
while(flag[0]); //进入区  
flag[1]=true; //进入区  
critical section; //临界区  
flag[1]=false; //退出区  
remainder section; //剩余区
```

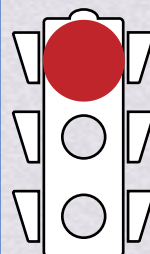
双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。

Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；

否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false



后教关注公众号「研大」
永久微信1951657888





软件同步机制

双标志先检查法

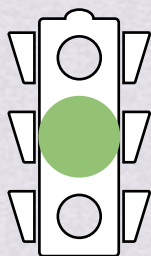
```
//进程P0  
while(flag[1]); //进入区  
flag[0]=true; //进入区  
critical section; //临界区  
flag[0]=false; //退出区  
remainder section; //剩余区
```

```
//进程P1  
while(flag[0]); //进入区  
flag[1]=true; //进入区  
critical section; //临界区  
flag[1]=false; //退出区  
remainder section; //剩余区
```

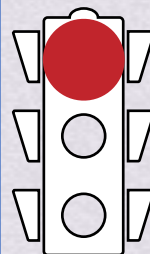
双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。

Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；

否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false



后教关注公众号「研大」
永久微信1951657888



- 单标志法
- 双标志后检查法
- peterson算法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制





软件同步机制

双标志先检查法

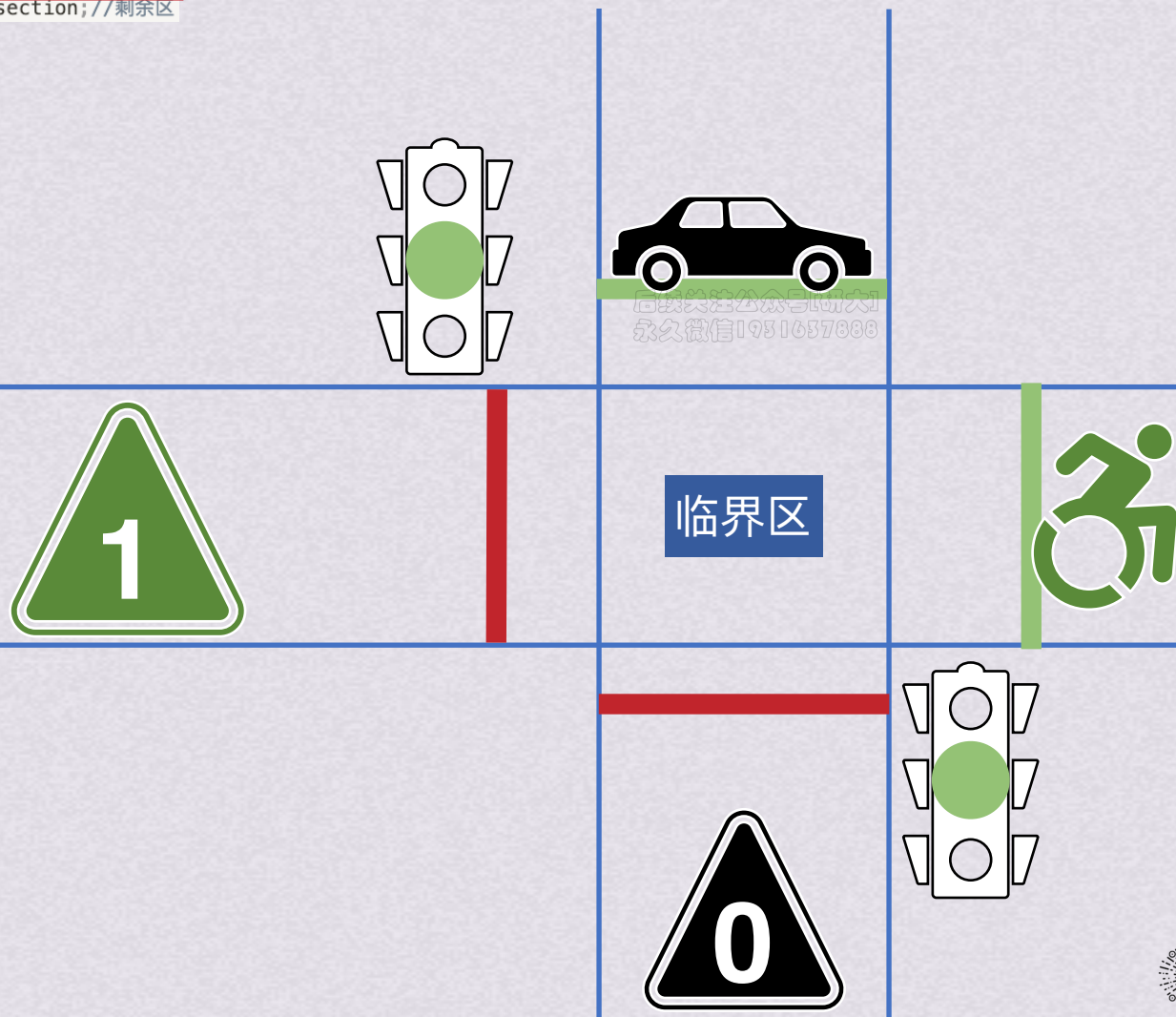
```
//进程P0  
while(flag[1]); //进入区  
flag[0]=true; //进入区  
critical section; //临界区  
flag[0]=false; //退出区  
remainder section; //剩余区
```

```
//进程P1  
while(flag[0]); //进入区  
flag[1]=true; //进入区  
critical section; //临界区  
flag[1]=false; //退出区  
remainder section; //剩余区
```

双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。

Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；

否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false



- 单标志法
- 双标志后检查法
- peterson算法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



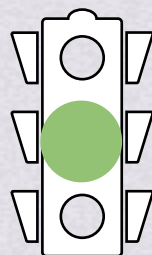


软件同步机制

双标志先检查法

双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false

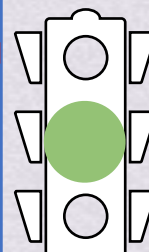
```
//进程P1
while(flag[0]); //进入区
flag[1]=true; //进入区
critical section; //临界区
flag[1]=false; //退出区
remainder section; //剩余区
```



后浪关注公众号「研大」
永久微信1951657888



临界区



```
//进程P0
while(flag[1]); //进入区
flag[0]=true; //进入区
critical section; //临界区
flag[0]=false; //退出区
remainder section; //剩余区
```




软件同步机制

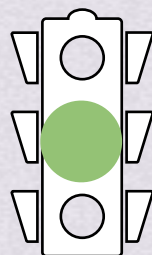
双标志先检查法

双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false

while循环相当于红绿灯机制

//进程P1

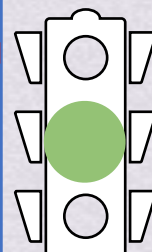
```
while(flag[0]); //进入区
flag[1]=true; //进入区
critical section; //临界区
flag[1]=false; //退出区
remainder section; //剩余区
```



后浪关注公众号[研大]
永久微信1951657888



临界区



//进程P0

```
while(flag[1]); //进入区
flag[0]=true; //进入区
critical section; //临界区
flag[0]=false; //退出区
remainder section; //剩余区
```

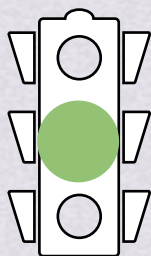


软件同步机制

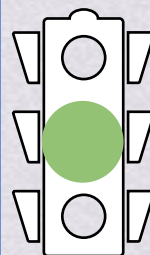
双标志先检查法

双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false

```
//进程P1
while(flag[0]); //进入区
flag[1]=true; //进入区
critical section; //临界区
flag[1]=false; //退出区
remainder section; //剩余区
```



临界区



while循环相当于红绿灯机制

设置对方的flag相当于改对方的红绿灯
但由于两个进程都是先检查flag[先看红绿灯]
初始时均为false[均为绿灯]
所以可能两个进程同时通过红绿灯
一起进入临界区

```
//进程P0
while(flag[1]); //进入区
flag[0]=true; //进入区
critical section; //临界区
flag[0]=false; //退出区
remainder section; //剩余区
```




软件同步机制

双标志先检查法

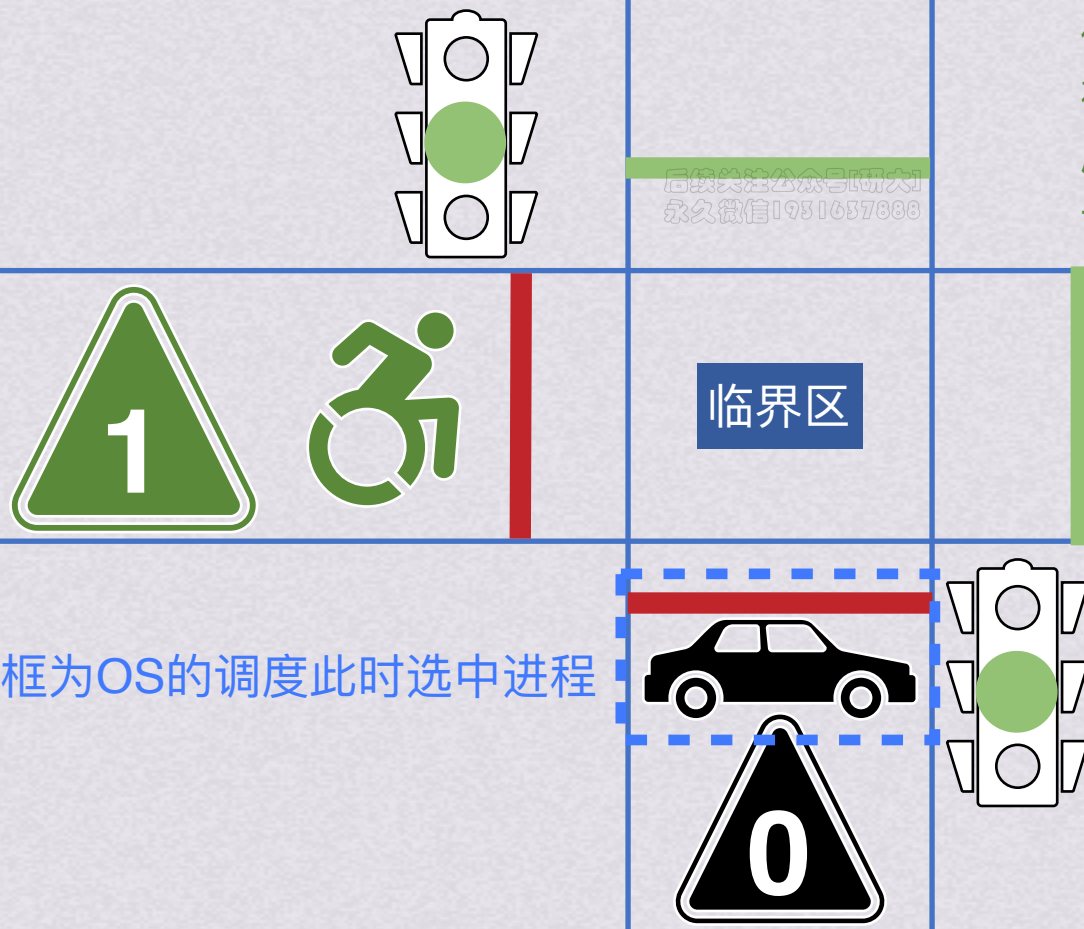
```
//进程P0
while(flag[1]); //进入区
flag[0]=true; //进入区
critical section; //临界区
flag[0]=false; //退出区
remainder section; //剩余区
```

```
//进程P1
while(flag[0]); //进入区
flag[1]=true; //进入区
critical section; //临界区
flag[1]=false; //退出区
remainder section; //剩余区
```

双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false

while循环相当于红绿灯机制

设置对方的flag相当于改对方的红绿灯
但由于两个进程都是先检查flag[先看红绿灯]
初始时均为false[均为绿灯]
所以可能两个进程同时通过红绿灯
一起进入临界区



蓝框为OS的调度此时选中进程

单标志法
双标志后检查法
peterson算法
同步互斥的基本概念
硬件同步机制
信号量机制





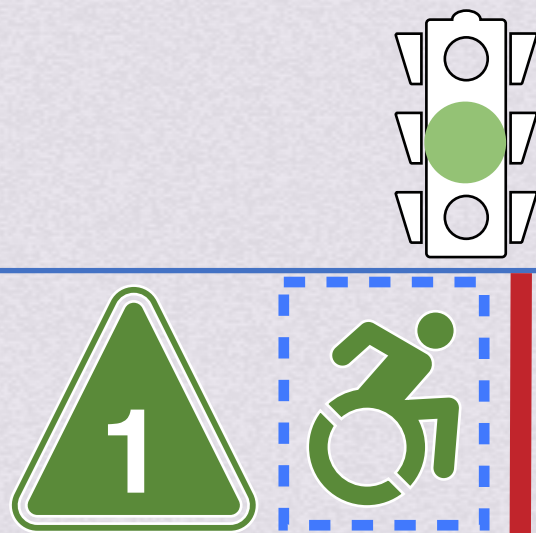
软件同步机制

双标志先检查法

```
//进程P0  
while(flag[1]); //进入区  
flag[0]=true; //进入区  
critical section; //临界区  
flag[0]=false; //退出区  
remainder section; //剩余区
```

```
//进程P1  
while(flag[0]); //进入区  
flag[1]=true; //进入区  
critical section; //临界区  
flag[1]=false; //退出区  
remainder section; //剩余区
```

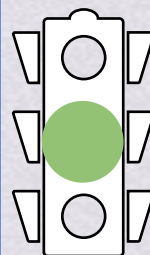
双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false



后浪关注公众号「研大」
永久微信1951657888

临界区

蓝框为OS的调度此时选中进程



while循环相当于红绿灯机制

设置对方的flag相当于改对方的红绿灯
但由于两个进程都是先检查flag[先看红绿灯]
初始时均为false[均为绿灯]
所以可能两个进程同时通过红绿灯
一起进入临界区

单标志法
双标志后检查法
peterson算法
同步互斥的基本概念
硬件同步机制
信号量机制





软件同步机制

双标志先检查法

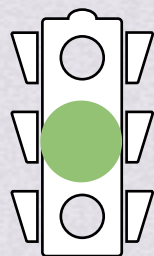
```
//进程P0
while(flag[1]); //进入区
flag[0]=true; //进入区
critical section; //临界区
flag[0]=false; //退出区
remainder section; //剩余区
```

```
//进程P1
while(flag[0]); //进入区
flag[1]=true; //进入区
critical section; //临界区
flag[1]=false; //退出区
remainder section; //剩余区
```

双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false

while循环相当于红绿灯机制

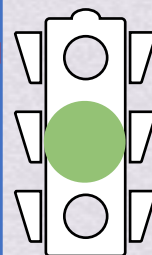
设置对方的flag相当于改对方的红绿灯
但由于两个进程都是先检查flag[先看红绿灯]
初始时均为false[均为绿灯]
所以可能两个进程同时通过红绿灯
一起进入临界区



后浪关注公众号「研大」
永久微信1951657888



蓝框为OS的调度此时选中进程



单标志法
双标志后检查法
peterson算法
同步互斥的基本概念
硬件同步机制
信号量机制





软件同步机制

双标志先检查法

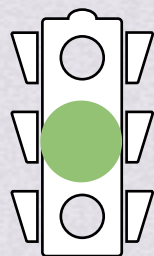
```
//进程P0
while(flag[1]); //进入区
flag[0]=true; //进入区
critical section; //临界区
flag[0]=false; //退出区
remainder section; //剩余区
```

```
//进程P1
while(flag[0]); //进入区
flag[1]=true; //进入区
critical section; //临界区
flag[1]=false; //退出区
remainder section; //剩余区
```

双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false

while循环相当于红绿灯机制

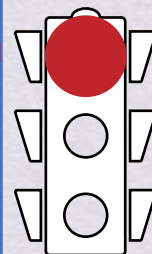
设置对方的flag相当于改对方的红绿灯
但由于两个进程都是先检查flag[先看红绿灯]
初始时均为false[均为绿灯]
所以可能两个进程同时通过红绿灯
一起进入临界区



后浪关注公众号「研大」
永久微信1951657888



蓝框为OS的调度此时选中进程



单标志法
双标志后检查法
peterson算法
同步互斥的基本概念
硬件同步机制
信号量机制





软件同步机制

双标志先检查法

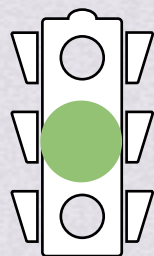
```
//进程P0  
while(flag[1]); //进入区  
flag[0]=true; //进入区  
critical section; //临界区  
flag[0]=false; //退出区  
remainder section; //剩余区
```

```
//进程P1  
while(flag[0]); //进入区  
flag[1]=true; //进入区  
critical section; //临界区  
flag[1]=false; //退出区  
remainder section; //剩余区
```

双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false

while循环相当于红绿灯机制

设置对方的flag相当于改对方的红绿灯
但由于两个进程都是先检查flag[先看红绿灯]
初始时均为false[均为绿灯]
所以可能两个进程同时通过红绿灯
一起进入临界区

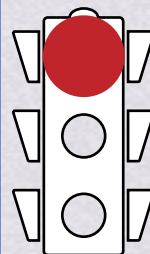


后浪关注公众号「研大」
永久微信1951657888



蓝框为OS的调度此时选中进程

0进程[汽车]此时已经检查过了红绿灯
虽然现在是红灯
但是之前检查的时候是绿灯，直接冲





软件同步机制

双标志先检查法

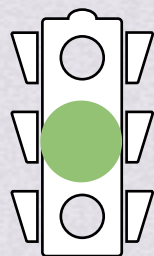
```
//进程P0
while(flag[1]); //进入区
flag[0]=true; //进入区
critical section; //临界区
flag[0]=false; //退出区
remainder section; //剩余区
```

```
//进程P1
while(flag[0]); //进入区
flag[1]=true; //进入区
critical section; //临界区
flag[1]=false; //退出区
remainder section; //剩余区
```

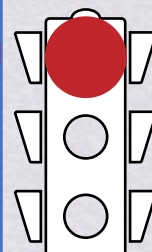
双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false

while循环相当于红绿灯机制

设置对方的flag相当于改对方的红绿灯
但由于两个进程都是先检查flag[先看红绿灯]
初始时均为false[均为绿灯]
所以可能两个进程同时通过红绿灯
一起进入临界区



后浪关注公众号「研大」
永久微信1951657888



蓝框为OS的调度此时选中进程

0进程[汽车]此时已经检查过了红绿灯
虽然现在为红灯
但是之前检查的时候是绿灯，直接冲

单标志法
双标志后检查法
peterson算法
同步互斥的基本概念
硬件同步机制
信号量机制





软件同步机制

双标志先检查法

双标志先检查法设置一个布尔型数组flag[2]，用来标记各个进程想进入临界区的意愿，flag[i]=true表示Pi想要进入临界区。Pi进入临界区前，先检查对方是否想进入临界区，若想，则等待；否则，将flag[i]置为true后，再进入临界区；当Pi退出临界区时，将flag[i]置为false

```
//进程P0
while(flag[1]); //进入区
flag[0]=true; //进入区
critical section; //临界区
flag[0]=false; //退出区
remainder section; //剩余区
```

```
//进程P1
while(flag[0]); //进入区
flag[1]=true; //进入区
critical section; //临界区
flag[1]=false; //退出区
remainder section; //剩余区
```

可能两个进程同时通过红绿灯

一起进入临界区





双标志先检查法





软件同步机制

双标志后检查法

- 单标志法
- 双标志先检查法
- peterson算法**
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制





软件同步机制

双标志后检查法

双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区

```
//进程P0
flag[0]=true;      //进入区
while(flag[1]);    //进入区
critical section;  //临界区
flag[0]=false;     //退出区
remainder section; //剩余区
```

```
//进程P1
flag[1]=true;      //进入区
while(flag[0]);    //进入区
critical section;  //临界区
flag[1]=false;     //退出区
remainder section; //剩余区
```





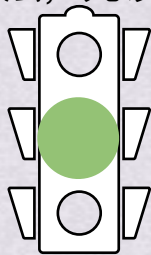
软件同步机制

双标志后检查法

```
//进程P0
flag[0]=true;    //进入区
while(flag[1]);  //进入区
critical section; //临界区
flag[0]=false;   //退出区
remainder section; //剩余区
```

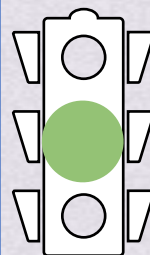
```
//进程P1
flag[1]=true;    //进入区
while(flag[0]);  //进入区
critical section; //临界区
flag[1]=false;   //退出区
remainder section; //剩余区
```

双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



临界区

改变策略为先踩线改红绿灯，再观察自己的红绿灯



单标志法
双标志先检查法
peterson算法
同步互斥的基本概念
硬件同步机制
信号量机制





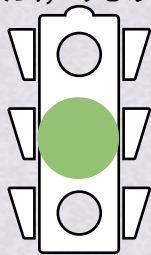
软件同步机制

双标志后检查法

```
//进程P0
flag[0]=true;    //进入区
while(flag[1]);  //进入区
critical section; //临界区
flag[0]=false;   //退出区
remainder section; //剩余区
```

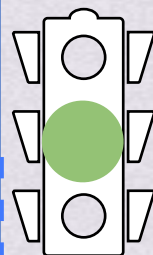
```
//进程P1
flag[1]=true;    //进入区
while(flag[0]);  //进入区
critical section; //临界区
flag[1]=false;   //退出区
remainder section; //剩余区
```

双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



临界区

改变策略为先踩线改红绿灯，再观察自己的红绿灯



单标志法
双标志先检查法
peterson算法
同步互斥的基本概念
硬件同步机制
信号量机制





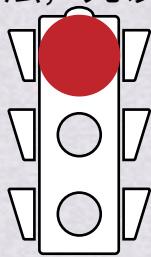
软件同步机制

双标志后检查法

```
//进程P0
flag[0]=true;    //进入区
while(flag[1]);  //进入区
critical section; //临界区
flag[0]=false;   //退出区
remainder section; //剩余区
```

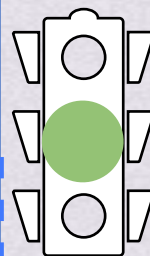
```
//进程P1
flag[1]=true;    //进入区
while(flag[0]);  //进入区
critical section; //临界区
flag[1]=false;   //退出区
remainder section; //剩余区
```

双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



临界区

改变策略为先踩线改红绿灯，再观察自己的红绿灯



单标志法
双标志先检查法
peterson算法
同步互斥的基本概念
硬件同步机制
信号量机制





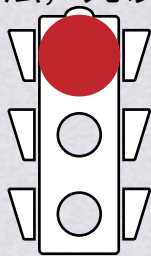
软件同步机制

双标志后检查法

```
//进程P0
flag[0]=true; //进入区
while(flag[1]); //进入区
critical section; //临界区
flag[0]=false; //退出区
remainder section; //剩余区
```

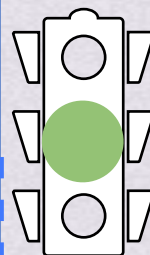
```
//进程P1
flag[1]=true; //进入区
while(flag[0]); //进入区
critical section; //临界区
flag[1]=false; //退出区
remainder section; //剩余区
```

双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



临界区

改变策略为先踩线改红绿灯，再观察自己的红绿灯





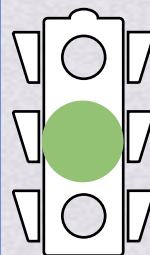
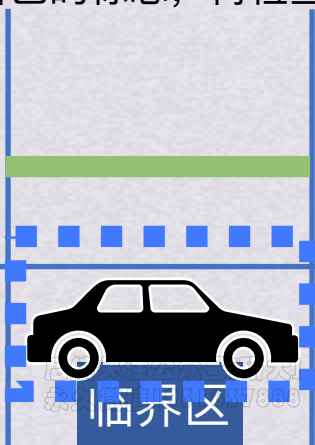
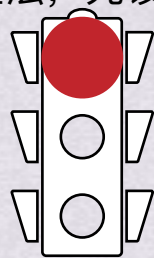
软件同步机制

双标志后检查法

```
//进程P0
flag[0]=true;    //进入区
while(flag[1]);  //进入区
critical section; //临界区
flag[0]=false;   //退出区
remainder section; //剩余区
```

```
//进程P1
flag[1]=true;    //进入区
while(flag[0]);  //进入区
critical section; //临界区
flag[1]=false;   //退出区
remainder section; //剩余区
```

双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



改变策略为先踩线改红绿灯，再观察自己的红绿灯

- 单标志法
- 双标志先检查法
- peterson算法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



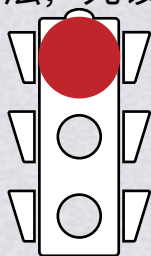
软件同步机制

双标志后检查法

```
//进程P0
flag[0]=true;    //进入区
while(flag[1]);  //进入区
critical_section; //临界区
flag[0]=false;   //退出区
remainder_section; //剩余区
```

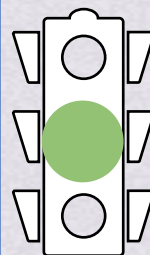
```
//进程P1
flag[1]=true;    //进入区
while(flag[0]);  //进入区
critical_section; //临界区
flag[1]=false;   //退出区
remainder_section; //剩余区
```

双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



临界区

改变策略为先踩线改红绿灯，再观察自己的红绿灯



- 单标志法
- 双标志先检查法
- peterson算法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制





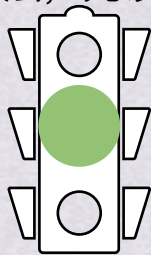
软件同步机制

双标志后检查法

```
//进程P0
flag[0]=true;    //进入区
while(flag[1]);  //进入区
critical_section; //临界区
flag[0]=false;   //退出区
remainder_section; //剩余区
```

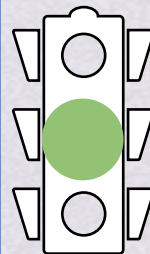
```
//进程P1
flag[1]=true;    //进入区
while(flag[0]);  //进入区
critical_section; //临界区
flag[1]=false;   //退出区
remainder_section; //剩余区
```

双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



临界区

改变策略为先踩线改红绿灯，再观察自己的红绿灯



- 单标志法
- 双标志先检查法
- peterson算法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制





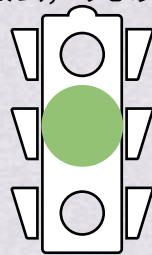
软件同步机制

双标志后检查法

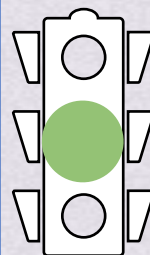
```
//进程P0
flag[0]=true;    //进入区
while(flag[1]);  //进入区
critical section; //临界区
flag[0]=false;   //退出区
remainder section; //剩余区
```

```
//进程P1
flag[1]=true;    //进入区
while(flag[0]);  //进入区
critical section; //临界区
flag[1]=false;   //退出区
remainder section; //剩余区
```

双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



临界区



改变策略为先踩线改红绿灯，再观察自己的红绿灯

单标志法
双标志先检查法
peterson算法
同步互斥的基本概念
硬件同步机制
信号量机制



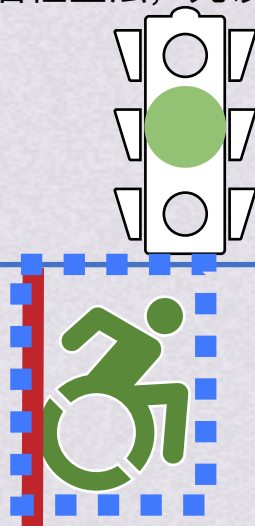
软件同步机制

双标志后检查法

```
//进程P0
flag[0]=true;    //进入区
while(flag[1]);  //进入区
critical section; //临界区
flag[0]=false;   //退出区
remainder section; //剩余区
```

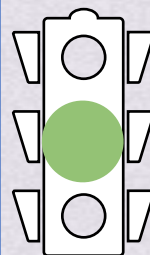
```
//进程P1
flag[1]=true;    //进入区
while(flag[0]);  //进入区
critical section; //临界区
flag[1]=false;   //退出区
remainder section; //剩余区
```

双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



临界区

改变策略为先踩线改红绿灯，再观察自己的红绿灯



- 单标志法
- 双标志先检查法
- peterson算法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制





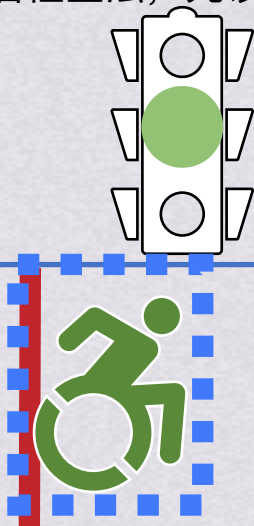
软件同步机制

双标志后检查法

```
//进程P0
flag[0]=true;    //进入区
while(flag[1]);  //进入区
critical section; //临界区
flag[0]=false;   //退出区
remainder section; //剩余区
```

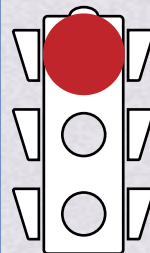
```
//进程P1
flag[1]=true;    //进入区
while(flag[0]);  //进入区
critical section; //临界区
flag[1]=false;   //退出区
remainder section; //剩余区
```

双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



临界区

改变策略为先踩线改红绿灯，再观察自己的红绿灯



- 单标志法
- 双标志先检查法
- peterson算法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制





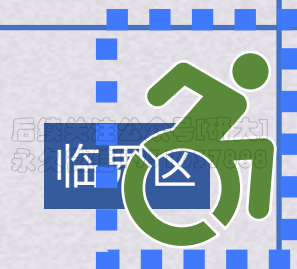
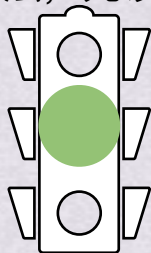
软件同步机制

双标志后检查法

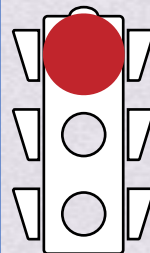
```
//进程P0
flag[0]=true;    //进入区
while(flag[1]);  //进入区
critical section; //临界区
flag[0]=false;   //退出区
remainder section; //剩余区
```

```
//进程P1
flag[1]=true;    //进入区
while(flag[0]);  //进入区
critical section; //临界区
flag[1]=false;   //退出区
remainder section; //剩余区
```

双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



改变策略为先踩线改红绿灯，再观察自己的红绿灯



- 单标志法
- 双标志先检查法
- peterson算法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



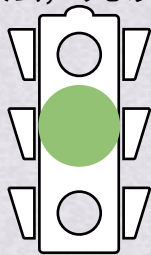
软件同步机制

双标志后检查法

```
//进程P0
flag[0]=true;    //进入区
while(flag[1]);  //进入区
critical section; //临界区
flag[0]=false;   //退出区
remainder section; //剩余区
```

```
//进程P1
flag[1]=true;    //进入区
while(flag[0]);  //进入区
critical section; //临界区
flag[1]=false;   //退出区
remainder section; //剩余区
```

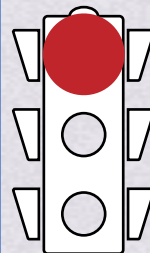
双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



临界区



改变策略为先踩线改红绿灯，再观察自己的红绿灯



- 单标志法
- 双标志先检查法
- peterson算法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



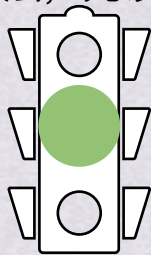
软件同步机制

双标志后检查法

```
//进程P0
flag[0]=true;    //进入区
while(flag[1]);  //进入区
critical section; //临界区
flag[0]=false;   //退出区
remainder section; //剩余区
```

```
//进程P1
flag[1]=true;    //进入区
while(flag[0]);  //进入区
critical section; //临界区
flag[1]=false;   //退出区
remainder section; //剩余区
```

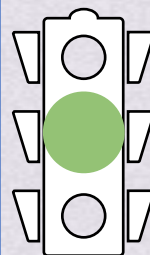
双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



临界区



改变策略为先踩线改红绿灯，再观察自己的红绿灯



单标志法
双标志先检查法
peterson算法
同步互斥的基本概念
硬件同步机制
信号量机制



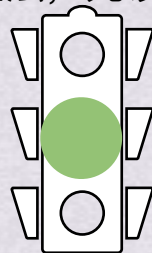
软件同步机制

双标志后检查法

```
//进程P0
flag[0]=true;    //进入区
while(flag[1]);  //进入区
critical section; //临界区
flag[0]=false;   //退出区
remainder section; //剩余区
```

```
//进程P1
flag[1]=true;    //进入区
while(flag[0]);  //进入区
critical section; //临界区
flag[1]=false;   //退出区
remainder section; //剩余区
```

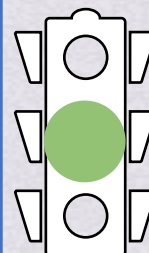
双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



临界区

改变策略为先踩线改红绿灯，再观察自己的红绿灯

但是不难发现如果两进程都改好标志[踩线]
再判断while[看红绿灯]的话
又会造成都想进入临界区而进不去的局面



单标志法
双标志先检查法
peterson算法
同步互斥的基本概念
硬件同步机制
信号量机制





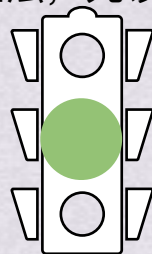
软件同步机制

双标志后检查法

```
//进程P0
flag[0]=true;    //进入区
while(flag[1]);  //进入区
critical section; //临界区
flag[0]=false;   //退出区
remainder section; //剩余区
```

```
//进程P1
flag[1]=true;    //进入区
while(flag[0]);  //进入区
critical section; //临界区
flag[1]=false;   //退出区
remainder section; //剩余区
```

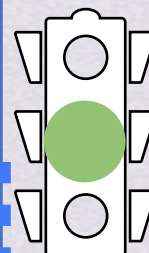
双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



临界区

改变策略为先踩线改红绿灯，再观察自己的红绿灯

但是不难发现如果两进程都改好标志[踩线]
再判断while[看红绿灯]的话
又会造成都想进入临界区而进不去的局面



单标志法
双标志先检查法
peterson算法
同步互斥的基本概念
硬件同步机制
信号量机制





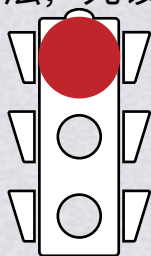
软件同步机制

双标志后检查法

```
//进程P0
flag[0]=true;    //进入区
while(flag[1]);  //进入区
critical section; //临界区
flag[0]=false;   //退出区
remainder section; //剩余区
```

```
//进程P1
flag[1]=true;    //进入区
while(flag[0]);  //进入区
critical section; //临界区
flag[1]=false;   //退出区
remainder section; //剩余区
```

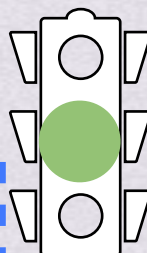
双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



临界区

改变策略为先踩线改红绿灯，再观察自己的红绿灯

但是不难发现如果两进程都改好标志[踩线]
再判断while[看红绿灯]的话
又会造成都想进入临界区而进不去的局面



- 单标志法
- 双标志先检查法
- peterson算法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



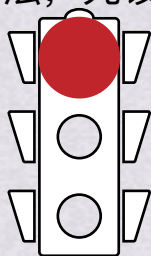
软件同步机制

双标志后检查法

```
//进程P0
flag[0]=true;    //进入区
while(flag[1]);  //进入区
critical section; //临界区
flag[0]=false;   //退出区
remainder section; //剩余区
```

```
//进程P1
flag[1]=true;    //进入区
while(flag[0]);  //进入区
critical section; //临界区
flag[1]=false;   //退出区
remainder section; //剩余区
```

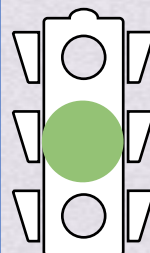
双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



临界区

改变策略为先踩线改红绿灯，再观察自己的红绿灯

但是不难发现如果两进程都改好标志[踩线]
再判断while[看红绿灯]的话
又会造成都想进入临界区而进不去的局面



单标志法
双标志先检查法
peterson算法
同步互斥的基本概念
硬件同步机制
信号量机制





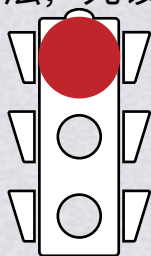
软件同步机制

双标志后检查法

```
//进程P0
flag[0]=true;    //进入区
while(flag[1]);  //进入区
critical section; //临界区
flag[0]=false;   //退出区
remainder section; //剩余区
```

```
//进程P1
flag[1]=true;    //进入区
while(flag[0]);  //进入区
critical section; //临界区
flag[1]=false;   //退出区
remainder section; //剩余区
```

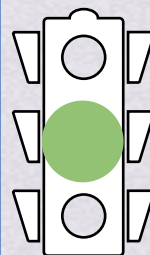
双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



临界区

改变策略为先踩线改红绿灯，再观察自己的红绿灯

但是不难发现如果两进程都改好标志[踩线]
再判断while[看红绿灯]的话
又会造成都想进入临界区而进不去的局面



单标志法
双标志先检查法
peterson算法
同步互斥的基本概念
硬件同步机制
信号量机制





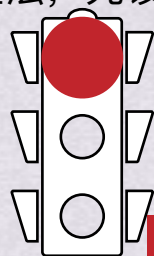
软件同步机制

双标志后检查法

```
//进程P0
flag[0]=true;    //进入区
while(flag[1]);  //进入区
critical section; //临界区
flag[0]=false;   //退出区
remainder section; //剩余区
```

```
//进程P1
flag[1]=true;    //进入区
while(flag[0]);  //进入区
critical section; //临界区
flag[1]=false;   //退出区
remainder section; //剩余区
```

双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



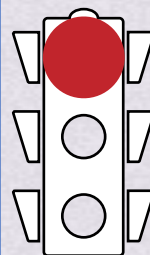
进程1观察自己的红绿灯是红色，进不去



临界区

改变策略为先踩线改红绿灯，再观察自己的红绿灯

但是不难发现如果两进程都改好标志[踩线]
再判断while[看红绿灯]的话
又会造成都想进入临界区而进不去的局面



单标志法
双标志先检查法
peterson算法
同步互斥的基本概念
硬件同步机制
信号量机制





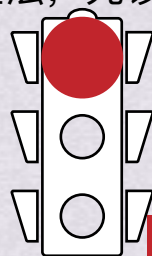
软件同步机制

双标志后检查法

```
//进程P0
flag[0]=true;    //进入区
while(flag[1]);  //进入区
critical section; //临界区
flag[0]=false;   //退出区
remainder section; //剩余区
```

```
//进程P1
flag[1]=true;    //进入区
while(flag[0]);  //进入区
critical section; //临界区
flag[1]=false;   //退出区
remainder section; //剩余区
```

双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区



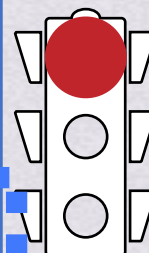
进程1观察自己的红绿灯是红色，进不去



临界区

改变策略为先踩线改红绿灯，再观察自己的红绿灯

但是不难发现如果两进程都改好标志[踩线]
再判断while[看红绿灯]的话
又会造成都想进入临界区而进不去的局面



进程0观察自己的红绿灯是红色，进不去



单标志法
双标志先检查法
peterson算法
同步互斥的基本概念
硬件同步机制
信号量机制





软件同步机制

双标志后检查法

双标志先检查法会先检查对方的标志再设置自己的，这俩操作无法一气呵成，于是两个进程可能同时进入临界区；所以想出了后检查法，先设置自己的标志，再检查对方的标志，若对方标志位true则等待；否则进入临界区

```
//进程P0
flag[0]=true;      //进入区
while(flag[1]);    //进入区
critical section;  //临界区
flag[0]=false;     //退出区
remainder section; //剩余区
```

```
//进程P1
flag[1]=true;      //进入区
while(flag[0]);    //进入区
critical section;  //临界区
flag[1]=false;     //退出区
remainder section; //剩余区
```

如果两进程都改好标志
再判断while的话

会造成都想进入临界区而进不去的局面
导致“饥饿”现象，违背“有限等待”准则



双标志后检查法





petererson算法

后续关注公众号「研才」
公众号ID: YC2008

- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制





软件同步机制

peterson算法

peterson算法结合了单标志法和双标志后检查法的思想，利用flag[]解决互斥访问问题，在利用turn解决饥饿问题。若双方都争着进入临界区，则可以让进程将进入临界区的机会让给对方。

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

后续关注公众号[研大]
永久微信1951657888

```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```





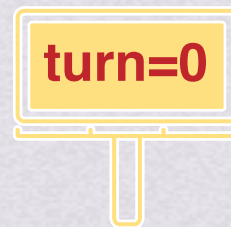
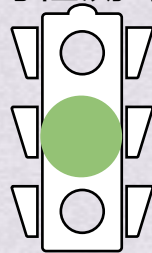
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入



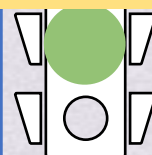
绿线功能是给别的进程亮绿灯

临界区

走到黑色点点处再判断能否通行

改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

黄线功能是让turn=别的进程



红线功能是给别的进程亮红灯



单标志法
双标志先检查法
双标志后检查法
同步互斥的基本概念
硬件同步机制
信号量机制





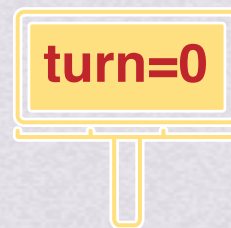
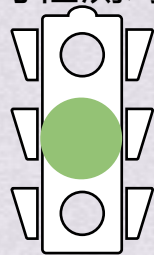
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

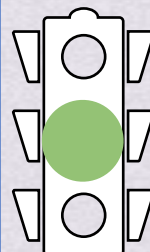
```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入



临界区



改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制





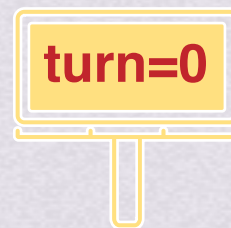
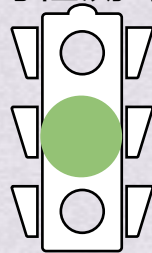
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

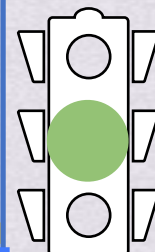
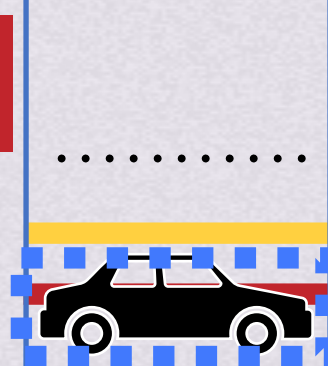
```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入



临界区



改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



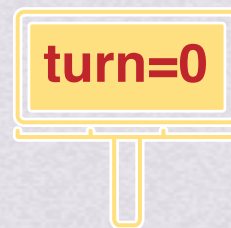
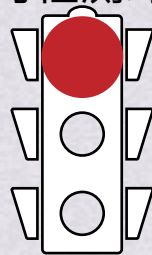
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

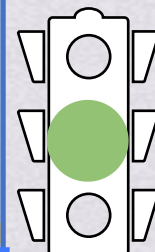
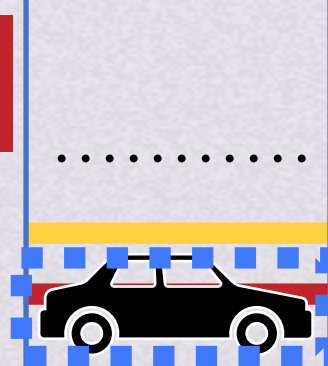
```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入



临界区



改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



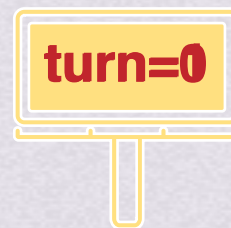
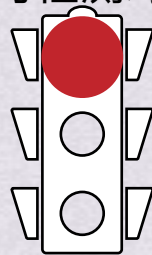
软件同步机制

```
//进程P0  
flag[0]=true;  
turn=1;  
while(flag[1]&&turn==1);  
critical section;  
flag[0]=false;  
remainder section;
```

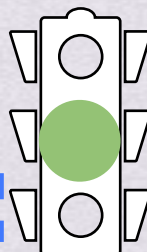
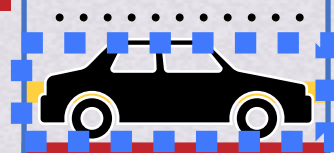
```
//进程P1  
flag[1]=true;  
turn=0;  
while(flag[0]&&turn==0);  
critical section;  
flag[1]=false;  
remainder section;
```

peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入



临界区



改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



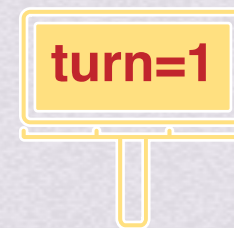
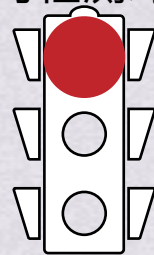
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

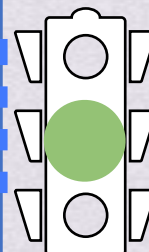
peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入



临界区

改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走



- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制





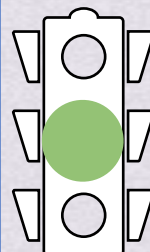
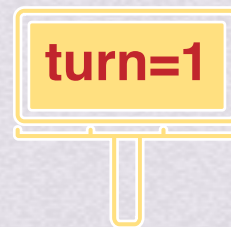
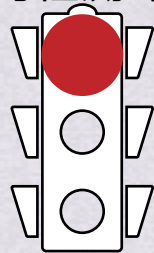
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入



改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



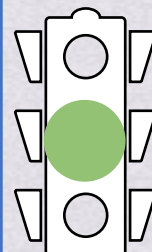
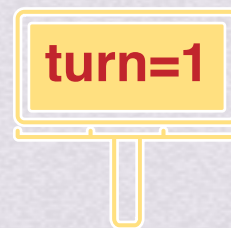
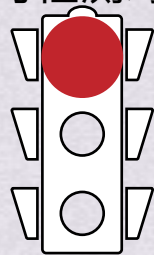
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入



.....



改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



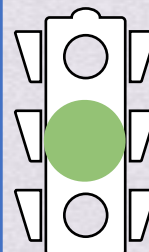
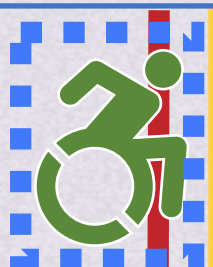
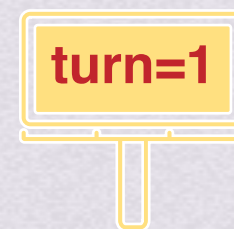
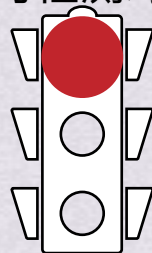
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入



改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



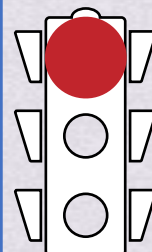
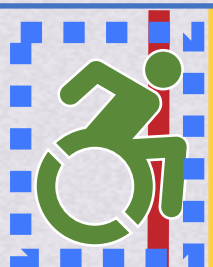
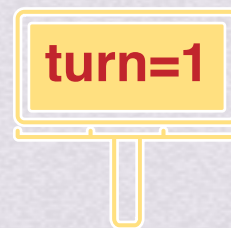
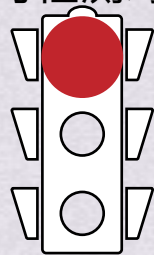
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入



改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



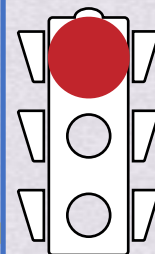
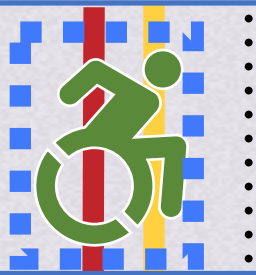
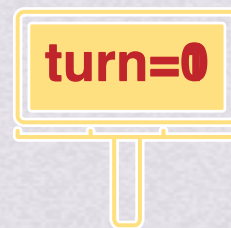
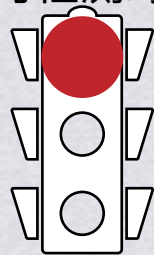
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入



.....



改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



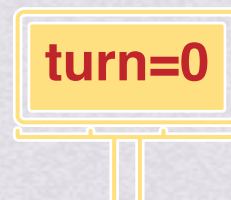
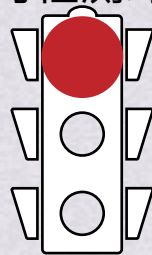
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入

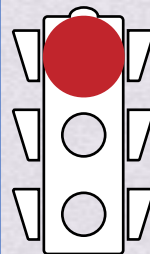


此处违反了让权等待，虽然1号进程无法进入临界区，但是此时他也不会释放CPU就一直卡在while里干等



改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

.....





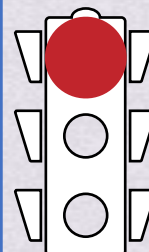
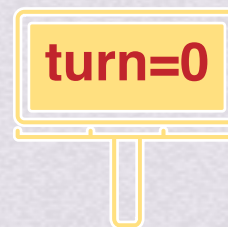
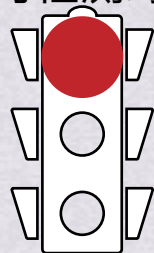
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入



改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



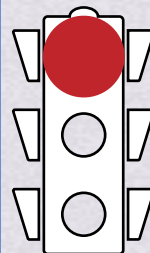
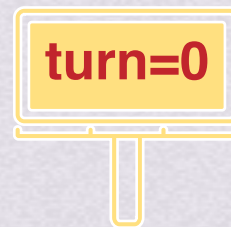
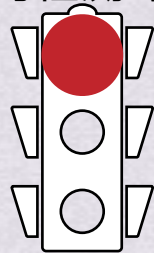
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入



改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



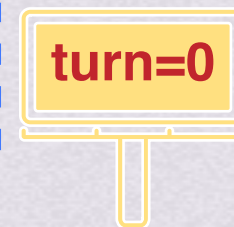
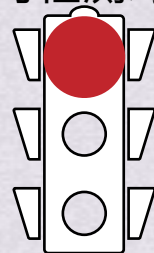
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

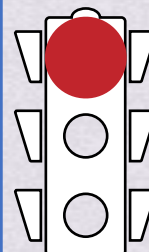
```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入



临界区



.....



改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



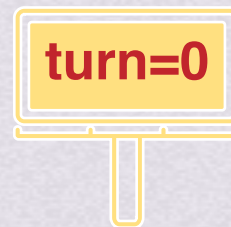
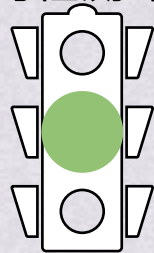
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

peterson算法

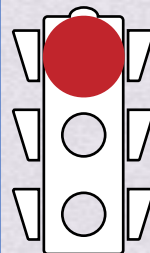
每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入



临界区

改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

.....



- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



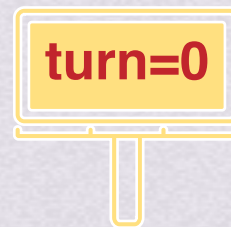
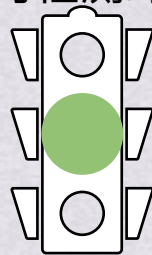
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

peterson算法

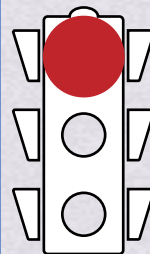
每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入



临界区

改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

.....



- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



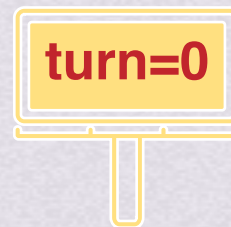
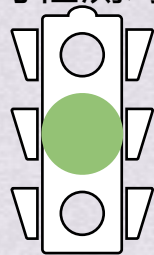
软件同步机制

```
//进程P0  
flag[0]=true;  
turn=1;  
while(flag[1]&&turn==1);  
critical section;  
flag[0]=false;  
remainder section;
```

```
//进程P1  
flag[1]=true;  
turn=0;  
while(flag[0]&&turn==0);  
critical section;  
flag[1]=false;  
remainder section;
```

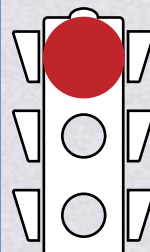
peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入



改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

.....



- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



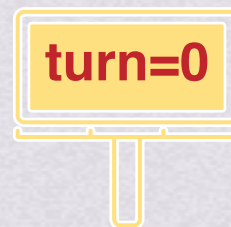
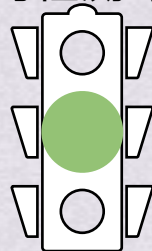
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

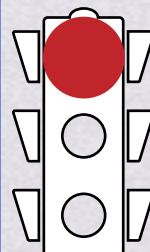
```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入



改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走



.....



- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



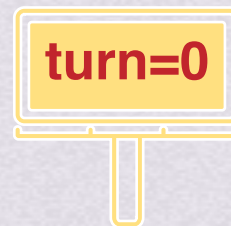
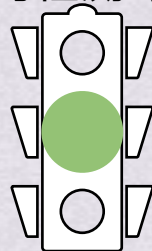
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入

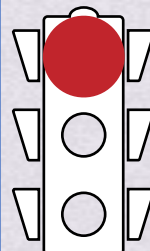


临界区



改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

.....



- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



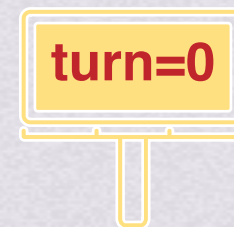
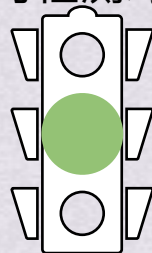
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入

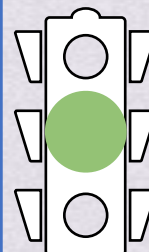


临界区



改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

.....



单标志法
双标志先检查法
双标志后检查法
同步互斥的基本概念
硬件同步机制
信号量机制



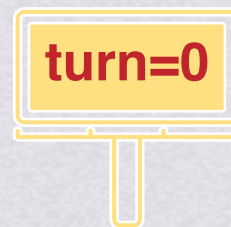
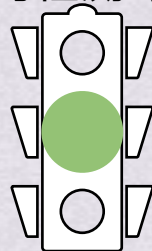
软件同步机制

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

peterson算法

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入

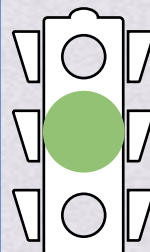


临界区



改变策略为只有看到红灯的同时turn不是自己，才被堵住
其他情况都直接走

.....



- 单标志法
- 双标志先检查法
- 双标志后检查法
- 同步互斥的基本概念
- 硬件同步机制
- 信号量机制



软件同步机制

peterson算法

peterson算法结合了单标志法和双标志后检查法的思想，利用flag[]解决互斥访问问题，在利用turn解决饥饿问题。若双方都争着进入临界区，则可以让进程将进入临界区的机会让给对方。

每个进程进入临界区之前，设置自己的flag标志，在设置允许进入turn标志，之后，再同时检测对方的flag和turn，以保证双方同时要求进入临界区时，只允许一个进程进入

```
//进程P0
flag[0]=true;
turn=1;
while(flag[1]&&turn==1);
critical section;
flag[0]=false;
remainder section;
```

后续关注公众号[研大]
永久微信1951657888

```
//进程P1
flag[1]=true;
turn=0;
while(flag[0]&&turn==0);
critical section;
flag[1]=false;
remainder section;
```

遵循了“空闲让进”“忙则等待”“有限等待”三个准则

未遵循“让权等待”准则

是几种软件算法里最好的

但是依然不够好





软件同步机制

单标志法

双标志先检查法

双标志后检查法

peterson算法

扫码关注公众号「研大」
永久微信 1951657888





软件同步机制





硬件同步机制

后像然然谷谷号四大
7344674月1954472936

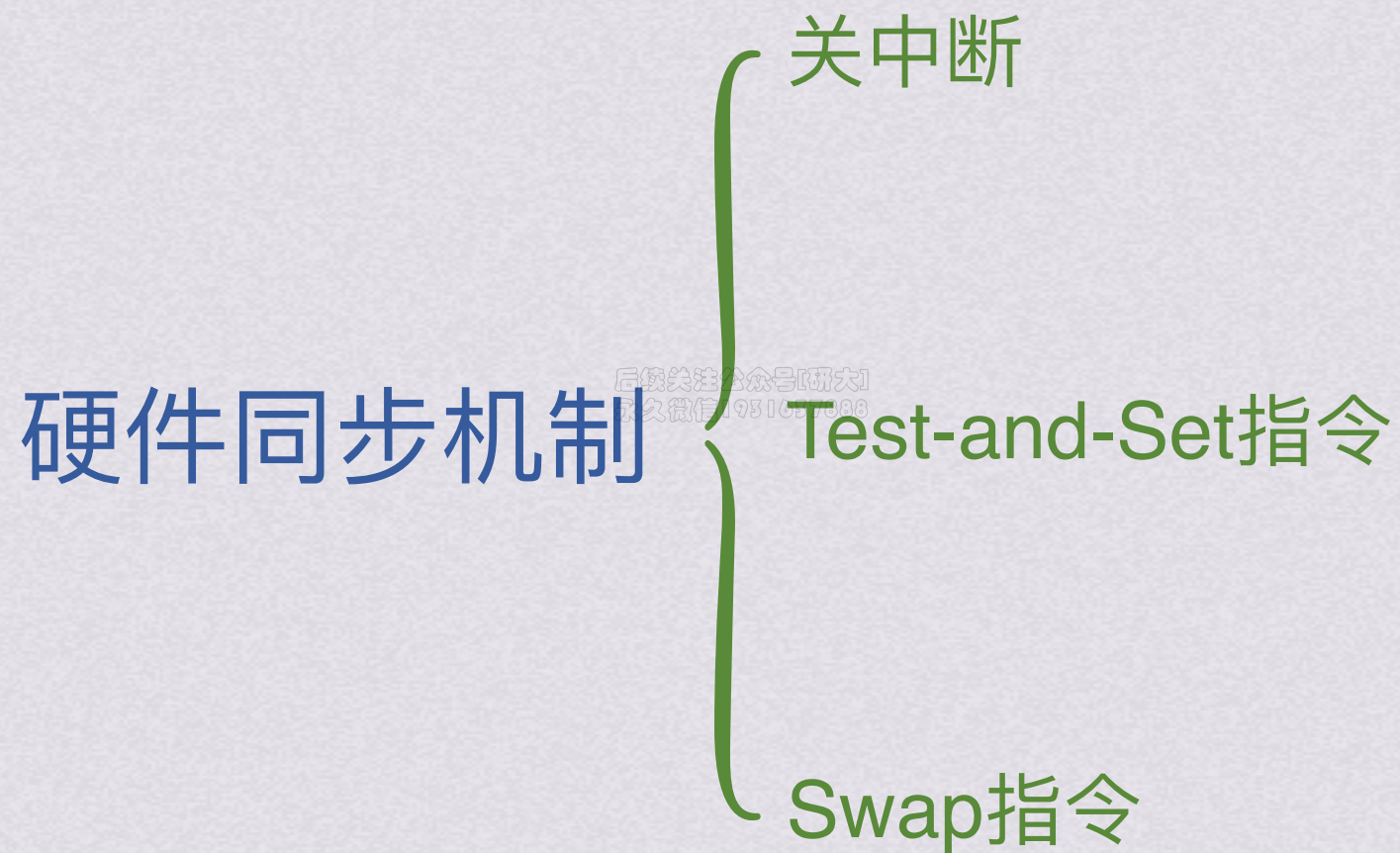




硬件同步机制

软件解决各个进程互斥进入临界区的问题有难度，而且有很大的局限性，所以计算机提供一些特殊的硬件指令来解决问题







硬件同步机制

关中断

有进程在临界区执行期间，计算机系统关中断，从而不会引发调度，也就不会有进程或线程切换

缺点：1.滥用关中断权利会导致严重后果

2.关中断时间过长，会影响系统效率

3.关中断方法不适用于多CPU系统，
在一个处理器上关中断不能防止进程在
其他处理器上执行相同临界代码

{ Test-and-Set指令
{ Swap指令

{ 同步互斥的基本概念
{ 软件同步机制
{ 信号量机制





关注公众号
BOK操作系统

关中断





Test-and-Set指令

后续关注公众号「码农大」
微信: 1951057888

{ 关中断
Swap指令

{ 同步互斥的基本概念
软件同步机制
信号量机制





硬件同步机制

Test-and-Set指令

```
//test and set
boolean TS(boolean *lock){
    boolean old;
    old = *lock;
    *lock = TRUE;
    return old;
}
```

```
do{
    ...
    while TS(&lock); /*do skip*/
    critical section;
    lock = FALSE;
    remainder section;
}while(TRUE);
```

后续关注公众号
永久微信1951657888

TS可以看作一个执行过程不可分割的函数过程（原语）

lock有两种状态：*lock=FALSE表示资源空闲

*lock=TRUE表示资源正被使用

{ 关中断
Swap指令

{ 同步互斥的基本概念
软件同步机制
信号量机制





硬件同步机制

Test-and-Set指令

```
//test and set
boolean TS(boolean *lock){
    boolean old;
    old = *lock;
    *lock = TRUE;
    return old;
}
```

```
do{
    ...
    while TS(&lock); /*do skip*/
    critical section;
    lock = FALSE;
    remainder section;
}while(TRUE);
```

后续关注公众号
永久微信1951657888

TS可以看作一个执行过程不可分割的函数过程（原语）

lock有两种状态：*lock=FALSE表示资源空闲

*lock=TRUE表示资源正被使用

使用TS管理临界区，要为每个临界资源设置一个lock
初值为FALSE，表示临界资源空闲。

进程进入临界区之前，先用TS测试lock，如果是FALSE，则可以
进入，并将TRUE值赋给lock，关闭了临界资源

{ 关中断
Swap指令

{ 同步互斥的基本概念
软件同步机制
信号量机制





Test-and-Set指令

后续关注公众号「码大」
微信ID: 951057888

{ 关中断
Swap指令

{ 同步互斥的基本概念
软件同步机制
信号量机制





Swap指令





硬件同步机制

Swap指令

```
//swap  
void swap(boolean *a, boolean *b){  
    boolean temp;  
    temp=*a;  
    *a=*b;  
    *b=temp;  
}
```

后续关注公众号[研大]
永久微信1951657888

称为对换指令，用于交换两个字的内容





硬件同步机制

Swap指令

```
//swap
void swap(boolean *a, boolean *b){
    boolean temp;
    temp=*a;
    *a=*b;
    *b=temp;
}
```

称为对换指令，用于交换两个字的内容

```
do{
    key = TRUE;
    do{
        swap(&lock, &key);
    }while(key!=FALSE);
    临界区操作;
    lock=FALSE;
    ...
}while(TRUE);
```

后续关注公众号[研大]
永久微信1951657888

为每个临界资源设置一个全局布尔变量lock，初值FALSE

利用上面的硬件指令完成进程互斥

但是这种方法也会造成访问进程不断进行测试，处于“忙等”状态，不符合“让权等待”原则

{ 关中断

{ Test-and-Set指令

{ 同步互斥的基本概念

{ 软件同步机制

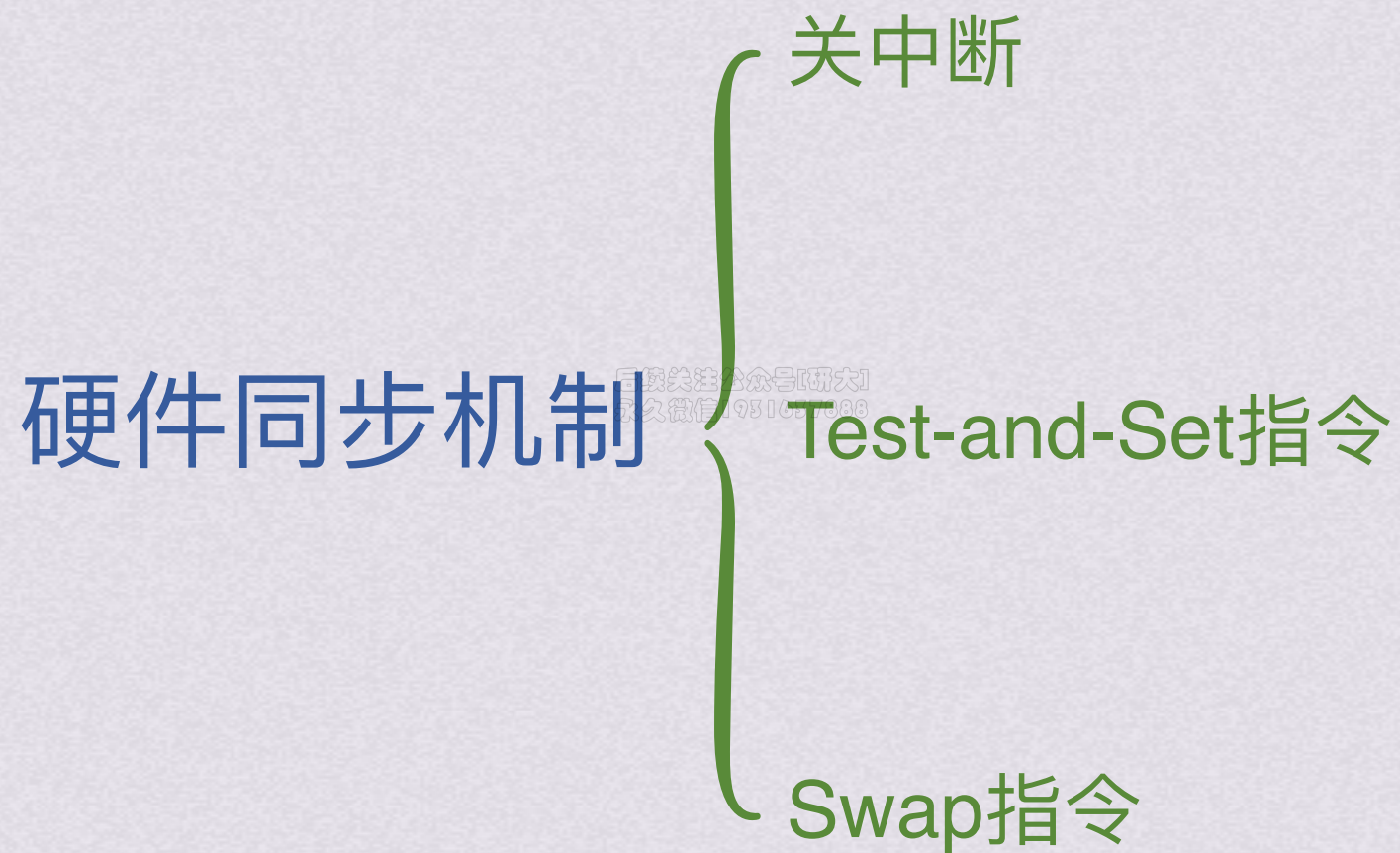
{ 信号量机制





Swap指令







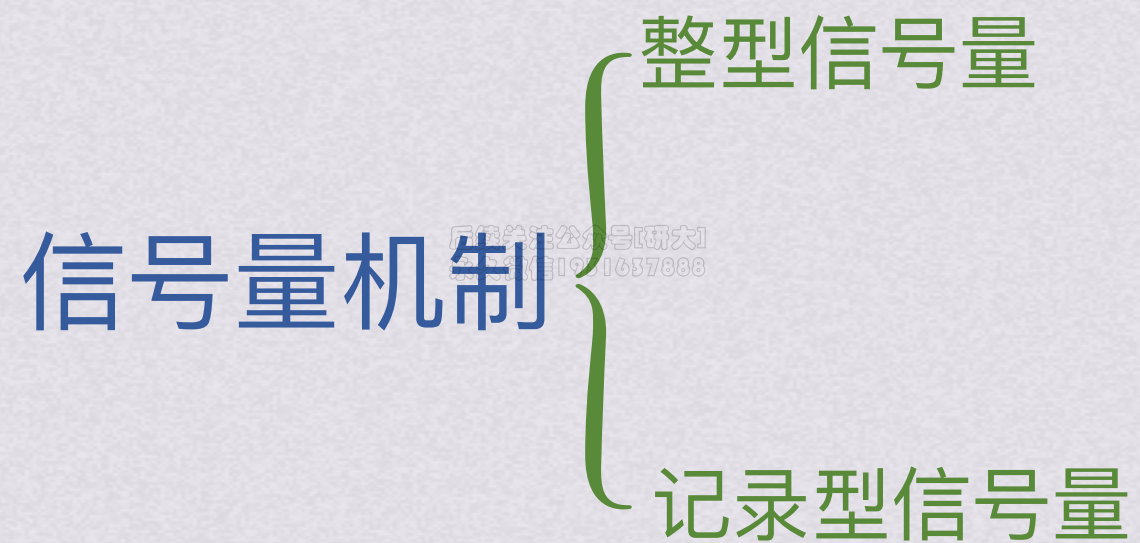
硬件同步机制





信号量机制







整型信号量

被定义为一个用于表示资源数目的整型量S，对这个整型信号量的操作只有三种：初始化、wait、signal

```
wait(S){  
    while(S<=0); //啥也不干  
    S--;  
}
```

后续关注公众号[研大]
永久微信1951657888

```
signal(S){  
    S++;  
}
```

wait和signal均为原子操作，执行时不可中断

当一个进程在修改某信号量时，没有其他进程可以同时对该信号量进行修改





整型信号量

被定义为一个用于表示资源数目的整型量 S ，对这个整型信号量的操作只有三种：初始化、`wait`、`signal`

```
wait(S){  
    while(S<=0); //啥也不干  
    S--;
```

后援关注微信公众号
永久微信1951657888

```
signal(S){  
    S++;  
}
```

这个过程会对信号量不断的进行测试，从而让进程处于“忙等”状态

`wait`和`signal`均为原子操作，执行时不可中断

当一个进程在修改某信号量时，没有其他进程可以同时对该信号量进行修改





整型信号量





记录型信号量

- { 整型信号量
- { 同步互斥的基本概念
- { 软件同步机制
- { 硬件同步机制





信号量机制

记录型信号量

这种机制是不存在“忙等”现象的进程同步机制，除了一个用于代表资源数目的整型变量value外，还需要增加一个进程链表L，用于链接所有等待该资源的进程。

```
typedef struct{
    int value;
    struct process *L;
}semaphore;
```

后续关注公众号「研大」

描述记录型信号量的结构体

```
void wait(semaphore S){
    S.value--;
    if(S.value<0){
        add this process to S.L;
        block(S.L);
    }
}
```

等价于申请资源的wait操作

```
void signal(semaphore S){
    S.value++;
    if(S.value<=0){
        remove a process P from S.L;
        wakeup(P);
    }
}
```

等价于释放资源的signal操作

{ 整型信号量
同步互斥的基本概念
软件同步机制
硬件同步机制





信号量机制

记录型信号量

这种机制是不存在“忙等”现象的进程同步机制，除了一个用于代表资源数目的整型变量value外，还需要增加一个进程链表L，用于链接所有等待该资源的进程。

```
void wait(semaphore S){  
    S.value--;  
    if(S.value<0){  
        add this process to S.L;  
        block(S.L);  
    }  
}
```

后续关注公众号「研大」
永久微信1951637888

等价于申请资源的wait操作

```
void signal(semaphore S){  
    S.value++;  
    if(S.value<=0){  
        remove a process P from S.L;  
        wakeup(P);  
    }  
}
```

等价于释放资源的signal操作

注意：wait操作等价于P操作

signal操作等价于V操作

只是两种不同的称呼，功能完全一致

$\text{wait}(A) = P(A)$

$\text{signal}(A) = V(A)$





记录型信号量

利用信号量实现进程互斥

设置一个互斥信号量 $\text{mutex}=1$ ，然后把各进程访问临界资源的临界区放在 $\text{wait}(\text{mutex})$ 和 $\text{signal}(\text{mutex})$ 之间

`semaphore mutex=0`

→

```
PA(){
    while(1){
        wait(mutex);
        临界区;
        signal(mutex);
        剩余区;
    }
}
```

后续关注公众号「研大」
永久微信1951657888

→

```
PB(){
    while(1){
        wait(mutex);
        临界区;
        signal(mutex);
        剩余区;
    }
}
```





记录型信号量

利用信号量实现进程互斥

设置一个互斥信号量 $\text{mutex}=1$ ，然后把各进程访问临界资源的临界区放在 $\text{wait}(\text{mutex})$ 和 $\text{signal}(\text{mutex})$ 之间

`semaphore mutex=0`

```
PA(){  
    while(1){  
        → wait(mutex);  
        临界区;  
        signal(mutex);  
        剩余区;  
    }  
}
```

后续关注公众号「研大」
永久微信1951657888

```
→ PB(){  
    while(1){  
        wait(mutex);  
        临界区;  
        signal(mutex);  
        剩余区;  
    }  
}
```





记录型信号量

利用信号量实现进程互斥

设置一个互斥信号量 $\text{mutex}=1$ ，然后把各进程访问临界资源的临界区放在 $\text{wait}(\text{mutex})$ 和 $\text{signal}(\text{mutex})$ 之间

$\text{semaphore mutex}=-1$ $S.\text{value}<0$ ，进程会调用block自我阻塞

```
PA(){
    while(1){
        → wait(mutex);
        临界区;
        signal(mutex);
        剩余区;
    }
}
```

后续关注公众号「研大」
永久微信1951657888

```
→ PB(){
    while(1){
        wait(mutex);
        临界区;
        signal(mutex);
        剩余区;
    }
}
```





记录型信号量

利用信号量实现进程互斥

设置一个互斥信号量 $\text{mutex}=1$ ，然后把各进程访问临界资源的临界区放在 $\text{wait}(\text{mutex})$ 和 $\text{signal}(\text{mutex})$ 之间

$\text{semaphore mutex}=-1$ $S.\text{value}<0$ ，进程会调用 block 自我阻塞

```
PA(){  
    while(1){  
        → wait(mutex);  
        临界区;  
        signal(mutex);  
        剩余区;  
    }  
}
```

后续关注公众号「研大」
永久微信1951657888

```
PB(){  
    while(1){  
        → wait(mutex);  
        临界区;  
        signal(mutex);  
        剩余区;  
    }  
}
```





记录型信号量

利用信号量实现进程互斥

设置一个互斥信号量 $\text{mutex}=1$ ，然后把各进程访问临界资源的临界区放在 $\text{wait}(\text{mutex})$ 和 $\text{signal}(\text{mutex})$ 之间

semaphore mutex=0

资源数++后 ≤ 0 ，证明有进程在等待该资源
应调用wakeup原语将S.L中第一个进程唤醒

```
PA(){
    while(1){
        → wait(mutex);
        临界区;
        signal(mutex);
        剩余区;
    }
}
```

后续关注公众号「研大」
永久微信1951657888

```
PB(){
    while(1){
        → wait(mutex);
        临界区;
        signal(mutex);
        剩余区;
    }
}
```





记录型信号量

利用信号量实现进程互斥

设置一个互斥信号量 $\text{mutex}=1$ ，然后把各进程访问临界资源的临界区放在 $\text{wait}(\text{mutex})$ 和 $\text{signal}(\text{mutex})$ 之间

`semaphore mutex=0`

资源数++后 ≤ 0 ，证明有进程在等待该资源
应调用`wakeup`原语将S.L中第一个进程唤醒

```
PA(){  
    while(1){  
        wait(mutex);  
        临界区;  
        signal(mutex);  
        剩余区;  
    }  
}
```

后续关注公众号「研大」
永久微信1951657888

```
PB(){  
    while(1){  
        wait(mutex);  
        临界区;  
        signal(mutex);  
        剩余区;  
    }  
}
```





记录型信号量

利用信号量实现进程互斥

设置一个互斥信号量 $\text{mutex}=1$ ，然后把各进程访问临界资源的临界区放在 $\text{wait}(\text{mutex})$ 和 $\text{signal}(\text{mutex})$ 之间

`semaphore mutex=0`

资源数++后 ≤ 0 ，证明有进程在等待该资源
应调用`wakeup`原语将S.L中第一个进程唤醒

```
PA(){  
    while(1){  
        wait(mutex);  
        临界区;  
        → signal(mutex);  
        剩余区;  
    }  
}
```

后续关注公众号「研大」
永久微信1951657888

```
PB(){  
    while(1){  
        → wait(mutex);  
        临界区;  
        signal(mutex);  
        剩余区;  
    }  
}
```





记录型信号量

利用信号量实现进程互斥

设置一个互斥信号量 $\text{mutex}=1$ ，然后把各进程访问临界资源的临界区放在 $\text{wait}(\text{mutex})$ 和 $\text{signal}(\text{mutex})$ 之间

`semaphore mutex=1`

资源数++后 ≤ 0 ，证明有进程在等待该资源
应调用`wakeup`原语将S.L中第一个进程唤醒

```
PA(){
    while(1){
        wait(mutex);
        临界区;
        → signal(mutex);
        剩余区;
    }
}
```

后续关注公众号「研大」
永久微信1951657888

```
PB(){
    while(1){
        wait(mutex);
        → 临界区;
        signal(mutex);
        剩余区;
    }
}
```





记录型信号量

利用信号量实现进程同步

设置一个同步信号量 $S=0$ ，让需要先执行的语句 $\text{signal}(S)$ ，后执行的 $\text{wait}(S)$

semaphore $S=-1$

→ **PC()**{
 先穿袜子;
 signal(S);
}

后续关注公众号[研大]
永久微信1951657888

→ **PD()**{
 wait(S);
 再穿鞋;
}





记录型信号量

利用信号量实现进程同步

设置一个同步信号量 $S=0$ ，让需要先执行的语句 $\text{signal}(S)$ ，后执行的 $\text{wait}(S)$

semaphore $S=-1$

→

```
PC(){  
    先穿袜子;  
    signal(S);  
}
```

后续关注公众号[研大]
永久微信1951657888

→

```
PD(){  
    wait(S);  
    再穿鞋;  
}
```





记录型信号量

利用信号量实现进程同步

设置一个同步信号量 $S=0$ ，让需要先执行的语句 $\text{signal}(S)$ ，后执行的 $\text{wait}(S)$

semaphore $S=-1$

→ **PC()**{
 先穿袜子;
 signal(S);
}

后续关注公众号「研大」
永久微信1951657888

→ **PD()**{
 wait(S);
 再穿鞋;
}





记录型信号量

利用信号量实现进程同步

设置一个同步信号量 $S=0$ ，让需要先执行的语句 $\text{signal}(S)$ ，后执行的 $\text{wait}(S)$

semaphore $S=0$

```
PC(){  
    先穿袜子;  
    signal(S);  
}
```

后续关注公众号[研大]
永久微信1951657888

```
PD(){  
    wait(S);  
    再穿鞋;  
}
```





记录型信号量

利用信号量实现进程同步

设置一个同步信号量 $S=0$ ，让需要先执行的语句 $\text{signal}(S)$ ，后执行的 $\text{wait}(S)$

semaphore $S=0$

```
PC(){  
    先穿袜子;  
    signal(S);  
}
```

后续关注公众号[研大]
永久微信1951657888

```
PD(){  
    wait(S);  
    再穿鞋;  
}
```





记录型信号量

利用信号量实现进程同步

设置一个同步信号量 $S=0$ ，让需要先执行的语句 $\text{signal}(S)$ ，后执行的 $\text{wait}(S)$

semaphore $S=0$

```
PC(){  
    先穿袜子;  
    → signal(S);  
}
```

后续关注公众号[研大]
永久微信1951657888

```
→ PD(){  
    wait(S);  
    再穿鞋;  
}
```





记录型信号量

利用信号量实现进程同步

设置一个同步信号量 $S=0$ ，让需要先执行的语句 $\text{signal}(S)$ ，后执行的 $\text{wait}(S)$

semaphore $S=0$

PC() {
 先穿袜子;
 → **signal(S);**
}

后续关注公众号「研大」
永久微信1951657888

PD() {
 wait(S);
 → 再穿鞋;
}

无论如何调度，都能做到先穿袜子再穿鞋！

